FONTOS: Minden stukinál ahol „!=” van oda annak helyére ≠ -t kell írni.

Vannak ebben hiányosságok, van amit teljesen nem tudtam hogyan csináljam meg.

1. A függvények aszimptotikus viselkedése

1.a

Az O(g) függvényhalmaz olyan f függvényekből áll, amiket

elég nagy n helyettesítési értékekre, megfelelő pozitív konstans szorzóval felülről becsül a g függvény:

O(g) = {f : ∃d ∈ P, hogy elég nagy n -ekre d ∗ g(n) ≥ f (n).}

f ∈ O(g) esetén azt mondjuk, hogy g aszimptotikus felső korlátja f -nek.

Az Ω(g) függvényhalmaz olyan f függvényekből áll, amiket

elég nagy n helyettesítési értékekre, megfelelelő pozitív konstans szorzóval alulról becsül a g függvény:

Ω(g) = {f : ∃c ∈ P, hogy elég nagy n -ekre c ∗ g(n) ≤ f (n).}

f ∈ Ω(g) esetén azt mondjuk, hogy g aszimptotikus alsó korlátja f -nek.

1.b

$$\Theta(g) = O(g) \cap \Omega(g)$$

1.c

g(n)=n^2, f(n)=$9n^2$ +24n + 16

Definíció szerint

$$c1 * g(n) \leq f(n) \leq c2 * g(n)$$

$$c1 * n^2 \leq 9n^2 + 24n + 16 \leq c2 * n^2$$

$$c1 \leq \frac{9n^2 + 24n + 16}{n^2} \leq c2$$

$$c1 \leq \frac{n^2}{n^2} * 9 + \frac{24}{n} + \frac{16}{n^2} \leq c2$$

$$c1 \leq 9 + 0 + 0 \leq c2$$

tehát a c1-nek kisebb egyenlőnek és c2 pedig nagyobb egyenlőnek kell lennie mint 9 ami lehetséges. Tehát az állítás igaz.

1.d

g(n)=n^n, f(n)=2^n

$c * g(n) \leq f(n)$

$c * 2^n \leq n^n$

$c \leq \frac{n^n}{2^n}$

Mivel $2^n \prec n^n (aszimptotikusan\ kisebb)$ ezért elég nagy n-re:
$c \leq \infty (\frac{n^n}{2^n} - nek\ végtelen\ a\ határértéke\ így)$, ami meg lehetséges, hisz tudunk olyan számot mondani, ami kisebb egyenlő, mint végtelen. Tehát az állítás igaz.

1.e

$g(n) = n^3, f(n) = 1000n^2 * (\lg n)$

$d * g(n) \geq f(n)$

$d * n^3 \geq 1000n^2 * (\lg n)$

$d \geq \frac{1000n^2 * (\lg n)}{n^3}$

$d \geq \frac{n3}{n3} * \frac{1000\frac{1}{n} * (\lg n)}{1}$

$d \geq 0$

Ilyen számot pedig tudunk mondani, tehát állításunk igaz.

2.

2.a -> 1.a-nál már leírva

2.b -> 2.b-nél már leírva

2.c

$g(n) = n^2, f(n) = 6n^2 - 5n - 4$

$c1 * n^2 \leq 6n^2 - 5n - 4 \leq c2 * n^2$

$c1 \leq \frac{6n^2 - 5n - 4}{n^2} \leq c2$

$c1 \leq \frac{n^2}{n^2} * 6 - \frac{5}{n} - \frac{4}{n^2} \leq c2$

$c1 \leq 6 - 0 - 0 \leq c2$

tehát a c1-nek kisebb egyenlőnek és c2 pedig nagyobb egyenlőnek kell lennie mint 6 ami lehetséges. Tehát az állítás igaz.

2.d

$g(n) = 2^n, f(n) = n!$

Mivel $2^{\mathrm{n}} \prec n!$ ezért elég nagy n-re:
n! aszimptotikusan felső korlátja lesz 2^{n}-nek. Tehát az állítás igaz.

2.e

$g(n) = n^2, f(n) = \mathrm{n} * \log n$

$c1 * n^2 \leq \mathrm{n} * \log n \leq c2 * n^2$

$c1 \leq \frac{\mathrm{n} * \log n}{n^2} \leq c2$

$c1 \leq \frac{n^2}{n^2} * \frac{1}{\mathrm{n}} * \log n \leq c2$

$c1 \leq 0 \leq c2$

Definícióból (Algo GY aszimptotika pdf) c1,c2> 0 és míg c2-re tudnánk számot mondani, c1-re nem mivel nincs olyan szám ami kisebb egyenlő lenne mint 0.
Matekosan: $\Theta(\mathrm{g}) = \mathrm{O}(\mathrm{g}) \cap \Omega(\mathrm{g}) = \emptyset, mert\ az\ Ordó\ nem\ teljes$ül.

2. Összehasonlító rendezések

1.a

Az összehasonlító rendezések: insertion, heap, merge, quick sort.
Osztályozás (*Nem tudom, hogy értette Ásványi de én így csináltam)*:

	mT(n)	AT(n)	MT(n)
insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
heap	$\Theta(n)$	$\Theta(n * \log n)$	$\Theta(n * \log n)$
merge	$\Theta(n * \log n)$	$\Theta(n * \log n)$	$\Theta(n * \log n)$
quick	$\Theta(n * \log n)$	$\Theta(n * \log n)$	$\Theta(n^2)$

1.b

Tetszőleges összehasonlító rendezésre MT(n)$\in \Omega(\mathrm{n} * \log n)$
(*Magyarázat: Tehát a MT bármelyik összehasonlító rendezést nézve legalább n*log n a műveletigénye)*

1.c

Bizonyítás

Elegendő alsó becslést adni a döntési fa h = MC(n) magasságára Jelölje a levelei számát l, akkor n! ≤l, továbbá l $\leq 2^h$, ugyanis egy h magasságú bináris fának legfeljebb 2^h levele van. Innét n! ≤l≤ 2^h, amiből n! $\leq 2^h$ adódik. Következésképpen:

MC(n) = h $\geq \log n! = \sum_{i=1}^{n} \log i \geq \sum_{i=\lceil \frac{n}{2} \rceil}^{n} \log i \geq \sum_{i=\lceil \frac{n}{2} \rceil}^{n} \log \left\lceil \frac{n}{2} \right\rceil \geq \left\lceil \frac{n}{2} \right\rceil * \log \left\lceil \frac{n}{2} \right\rceil \geq \frac{n}{2} *$
$\log \frac{n}{2} = \frac{n}{2} * (\log n - \log 2) = \frac{n}{2} * (\log n - 1) = \frac{n}{2} * \log n - \frac{n}{2} \in \Omega(n * \log n)$

A $\sum_{i=\lceil \frac{n}{2} \rceil}^{n} \log \left\lceil \frac{n}{2} \right\rceil \geq \left\lceil \frac{n}{2} \right\rceil * \log \left\lceil \frac{n}{2} \right\rceil$ egyenlőtlenséghez elegendő belátni, hogy az összegnek legalább $\left\lceil \frac{n}{2} \right\rceil$ tagja van. A tagok száma nyilván $n - \left(\left\lceil \frac{n}{2} \right\rceil - 1\right) = \left(n - \left\lceil \frac{n}{2} \right\rceil\right) + 1 = \left\lfloor \frac{n}{2} \right\rfloor + 1$, ami ha n páratlan szám, éppen $\left\lceil \frac{n}{2} \right\rceil$, ha pedig n páros, akkor $\left\lceil \frac{n}{2} \right\rceil + 1$

1.d

Ez a tétel jelentősége az, hogy meghatározza az összehasonlító rendezési algoritmusok hatékonysági határát és lehetővé teszi a rendezési algoritmusok összehasonlítását egymással

2.a -> 1.a-nál már leírva

2.b Tétel: Tetszőleges rendező algoritmusra mT(n) ∈ $\Omega(n)$

Bizonyítás:

Világos, hogy tetszőleges helyes rendező algoritmusoknak mind az n rendezendő elem értékét ellenőriznie kell, és mindegyik alprogram hívás és ciklusiteráció is csak korlátos számú elemet ellenőríz. Legyen ezeknek a korlátoknak a maximuma k. Akkor
$mT(n) * k \geq n => mT(n) = \frac{1}{k} * n =>$ mT(n) ∈ $\Omega(n)$

2.c

	mS(n)	MS(n)
insertion	$\Theta(1)$	$\Theta(1)$
heap	$\Theta(1)$	$\Theta(1)$
merge	$\Theta(n)$	$\Theta(n)$
quick	$\Theta(\log n)$	$\Theta(n)$

2.d

	mS(n)	MS(n)
insertion	$\Theta(1)$	$\Theta(1)$
heap	–	–
merge	$\Theta(\log n)$	$\Theta(\log n)$
quick	$\Theta(\log n)$	$\Theta(n)$

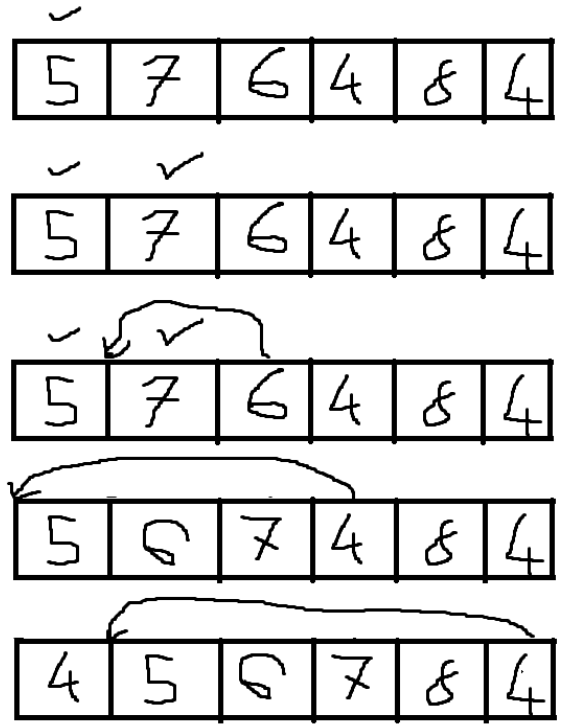
Insertion sortot, mert az tűnik a legoptimálisabbnak egyirányú listákra.

2.e

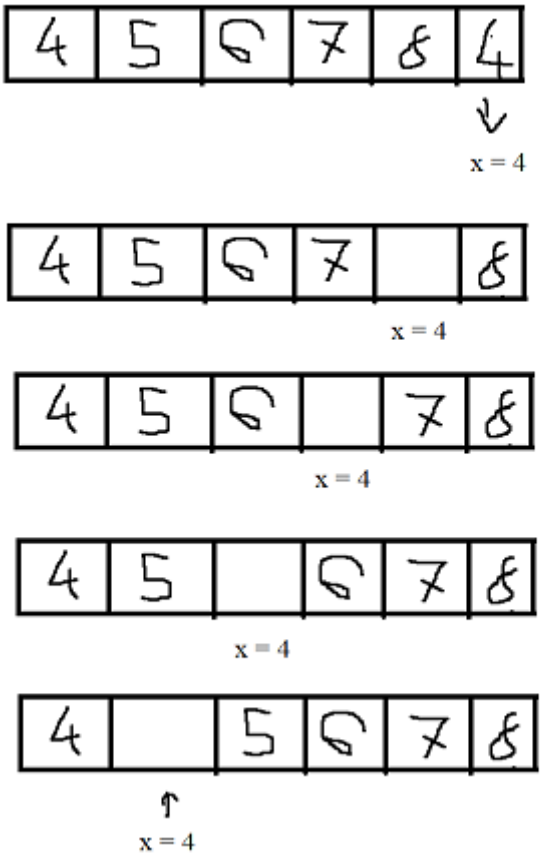
Quick sortot, mert előrendezett, kétirányú láncolt listákon való alkalmazása esetén nem szükséges újra létrehozni és mozgatni az elemeket és a magas szintű párhuzamosítást tesz lehetővé.

2.1 Beszuró rendezés

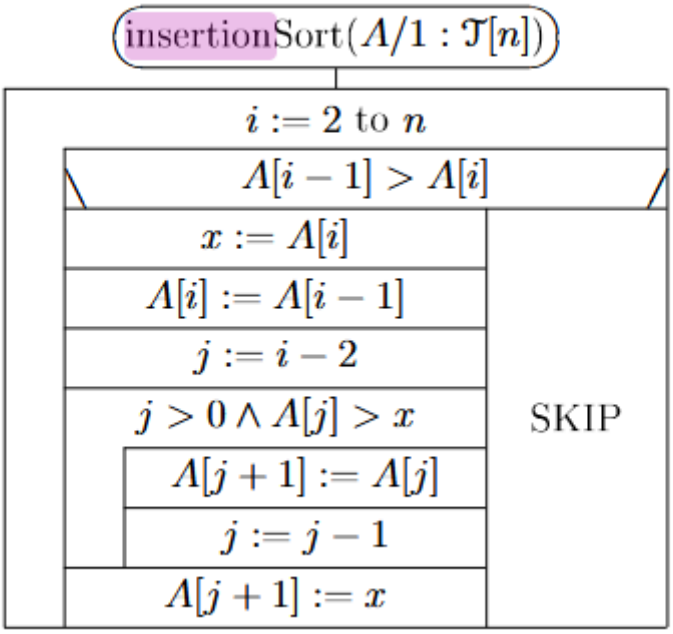
1.a



Az utolsó részletezése:



1.b



1.c

A ciklus mindig n-szer fut le, az azon belüli ciklus legfeljebb n-1-szer, tehát ha a tömb rendezve van, akkor az if-nél folyamatosan skippel, így a belső ciklusba soha nem lép be így a mT(n) = 1+(n-1) = n

MT(n) = 1+(n-1) + $\sum_{i=2}^{n}(i-2) = n + \sum_{k=0}^{n-2} k = n + \frac{(n-1)*(n-2)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n + 1$

1.d

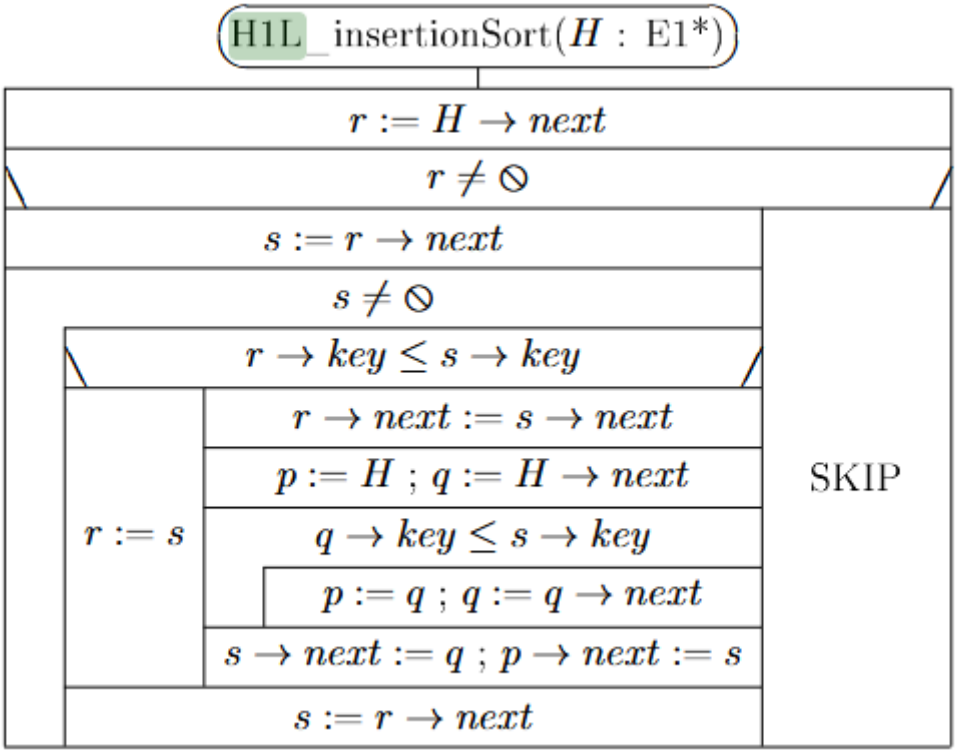
mT(n) =$\Theta(n)$

MT(n) = $\Theta(n^2)$

2.a

Egy rendezést akkor nevezünk stabilnak, ha nem változtatja meg az egyenlő kulcsú elemek egymáshoz viszonyított sorrendjét.

2.b



2.c

A belső ciklusban a q->key ≤s-> key feltétellel

2.d

Ugyanakkora, mint a tömbökre

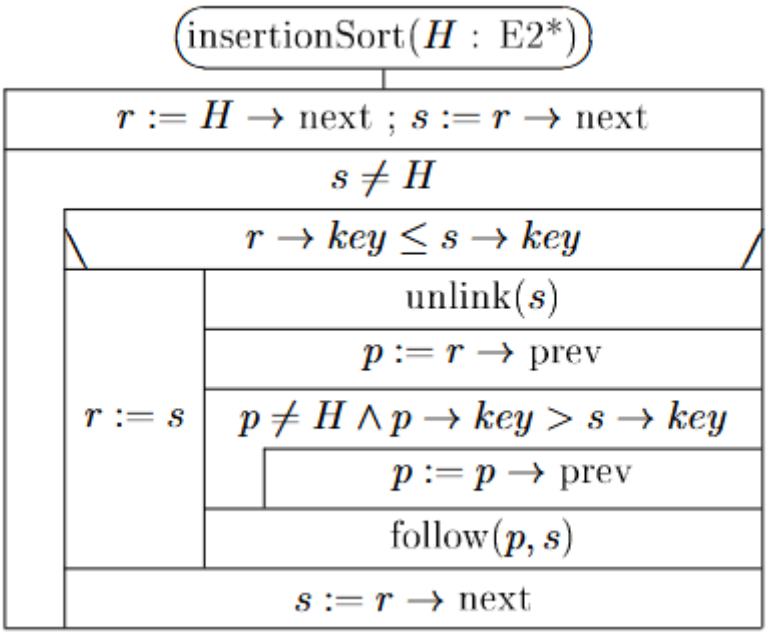
mT(n) = 1 + (n-1) = n

MT(n) = $\frac{1}{2}n^2 - \frac{1}{2}n + 1$

A két ciklus miatt lesz ugyanannyi a műveletigény

3.a -> 2.a-nál már leírva

3.b



E2
+prev, next : E2* // refer to the previous and next neighbour or be **this** +key : $\mathcal{T}$
+ E2() { prev := next := **this** }

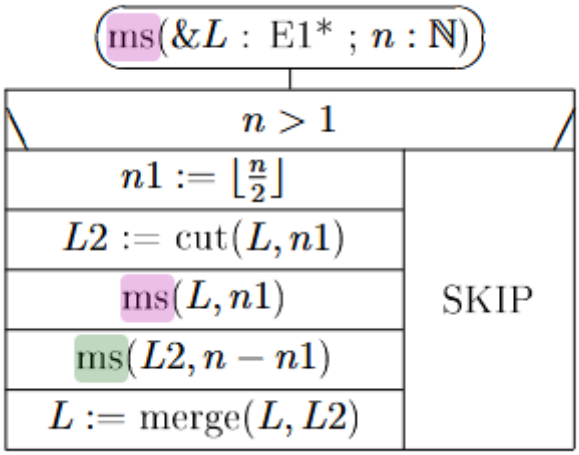
3.c A belső ciklus feltételével, ami $P \neq H \ \wedge p \rightarrow key >$s-> key

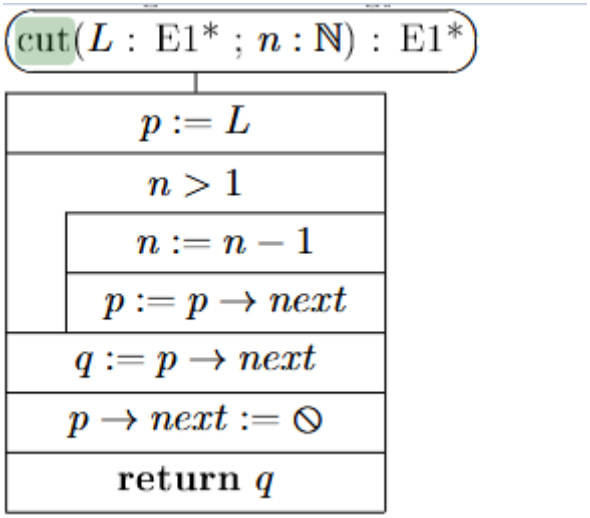
3.d ->2.d-nél már leírva

2.2 Összefésülő rendezés

1.a mergesort lejátszás

1.b





1.c

5.1.1. A merge eljárás műveletigénye

A merge(A, u, m, v) eljárás műveletigényének meghatározásához bevezetjük az $l = v - u$ jelölést. A merge eljárást csak akkor hívjuk meg, ha $u < v - 1$, azaz $l \geq 2$. $mT_{merge}(l)$ az eljárás minimális, $MT_{merge}(l)$ a maximális műveletigénye. Most csak 1 eljáráshívás van + az 1. ciklus $\lfloor l/2 \rfloor$ iterációja + a 2. és 3. ciklus iterációi: a Z tömbben lévő $\lfloor l/2 \rfloor$ elemet biztosan visszamásoljuk az A-ba, ami legalább $\lfloor l/2 \rfloor$ iteráció. Ha viszont a 2. és a 3. ciklus mindegyik elemet átmásolja, az összesen a maximális l iteráció. Innét

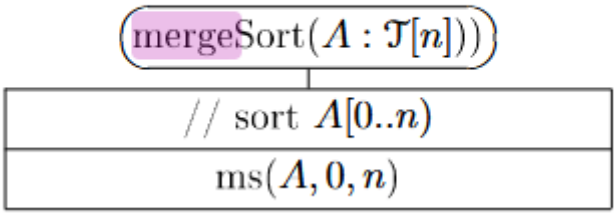
$mT_{merge}(l) \geq 1 + \lfloor \frac{l}{2} \rfloor + \lfloor \frac{l}{2} \rfloor \geq \lceil \frac{l}{2} \rceil + \lfloor \frac{l}{2} \rfloor = l$, valamint

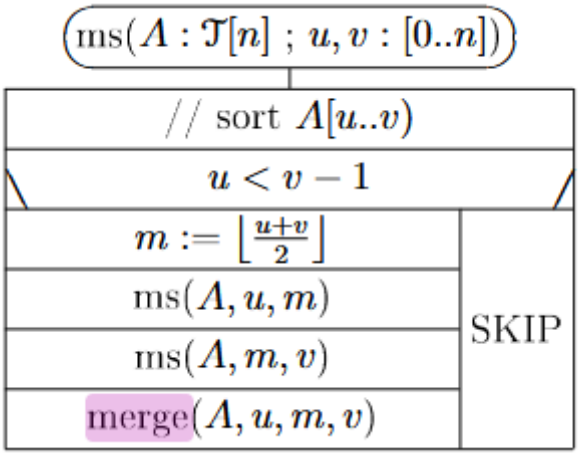
$MT_{merge}(l) \leq 1 + \lfloor \frac{l}{2} \rfloor + l \leq 2l$, ahonnét

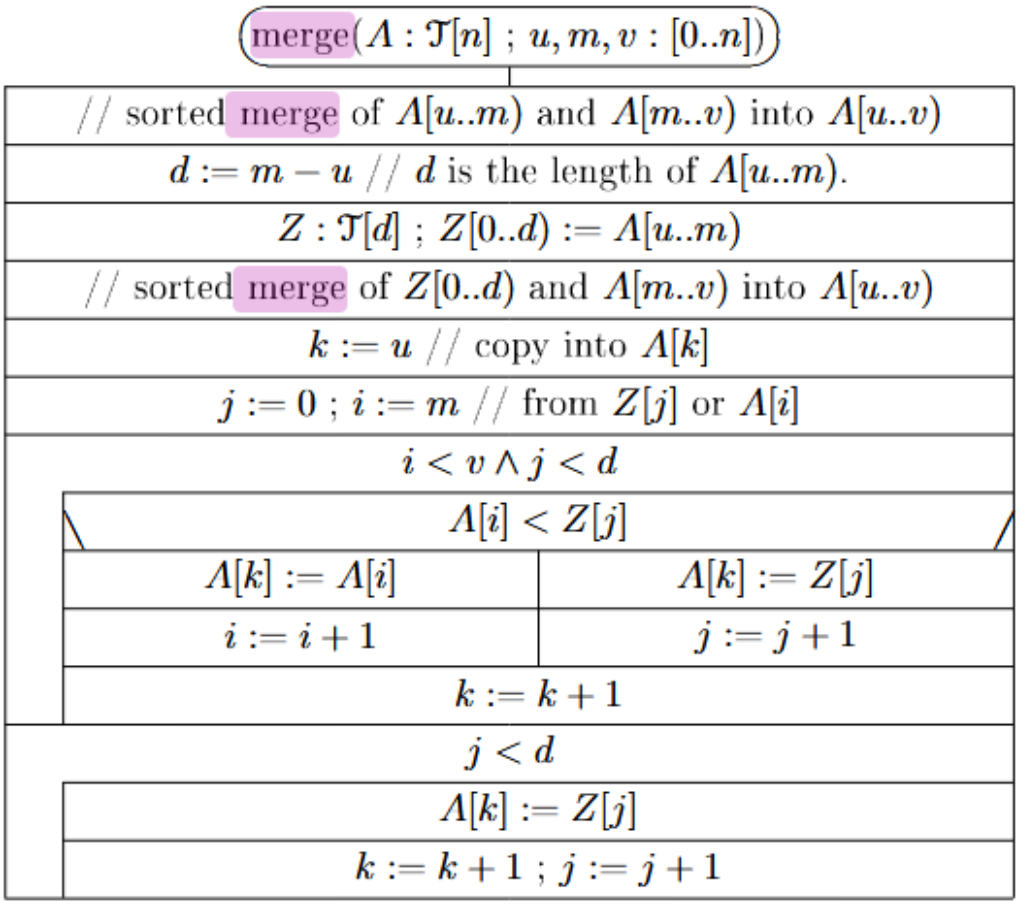
$$l \leq mT_{merge}(l) \leq MT_{merge}(l) \leq 2l$$

(Innét $mT_{merge}(l), MT_{merge}(l) \in \Theta(l)$ adódik.)

2.a


mergeSort($A : \mathcal{T}[n]$))
// sort $A[0..n)$
ms$(A, 0, n)$


ms$(A : \mathcal{T}[n]$; $u, v : [0..n])$
// sort $A[u..v)$
$u < v - 1$
$m := \lfloor \frac{u+v}{2} \rfloor$
ms(A, u, m)
ms(A, m, v)
merge(A, u, m, v)
SKIP

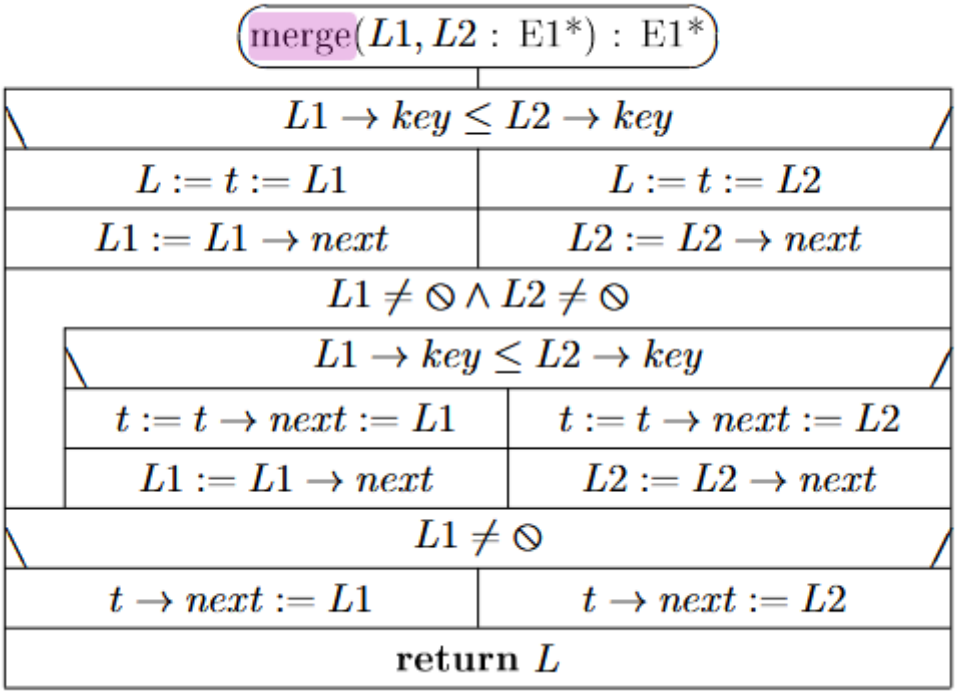

merge$(A : \mathcal{T}[n]$; $u, m, v : [0..n])$
// sorted merge of $A[u..m)$ and $A[m..v)$ into $A[u..v)$
$d := m - u$ // d is the length of $A[u..m)$.
$Z : \mathcal{T}[d]$; $Z[0..d) := A[u..m)$
// sorted merge of $Z[0..d)$ and $A[m..v)$ into $A[u..v)$
$k := u$ // copy into $A[k]$
$j := 0$; $i := m$ // from $Z[j]$ or $A[i]$
$i < v \wedge j < d$
$A[i] < Z[j]$
$A[k] := A[i]$
$A[k] := Z[j]$
$i := i + 1$
$j := j + 1$
$k := k + 1$
$j < d$
$A[k] := Z[j]$
$k := k + 1$; $j := j + 1$

2.b

Minimális, átlagos és legrosszabb esetben is a műveletigény $\Theta(n * \log n)$, mert az algoritmus a felosztást és az összefésülést végrehajtja minden elemre. Tárigény:
Tömbre: $\Theta(n)$
Láncolt listára: $\Theta(\log n)$

3.a mergesort

3.b

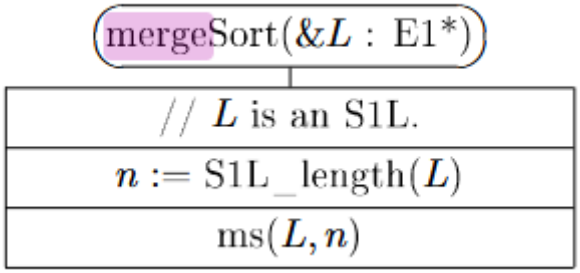


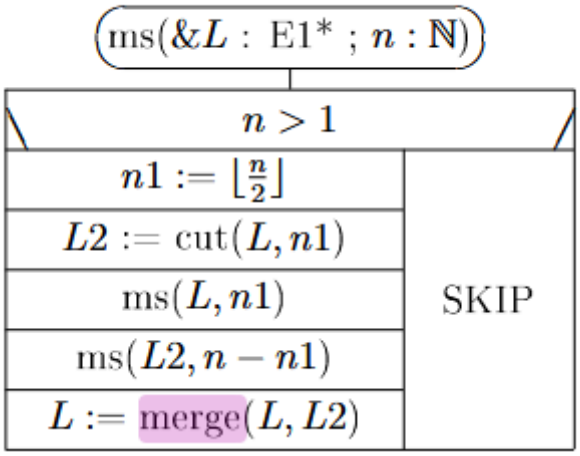
3.c

Változó a L1 és L2 hosszútól függ. Ha L1 hossza n és L2 hossza m akkor a merge műveletigénye: $\Theta(n + m)$

4.a mergesort

4.b





4.c

$mT_{MS}(n) = \Theta(n * \log n)$

$AT_{MS}(n) = \Theta(n * \log n)$

$MT_{MS}(n) = \Theta(n * \log n)$

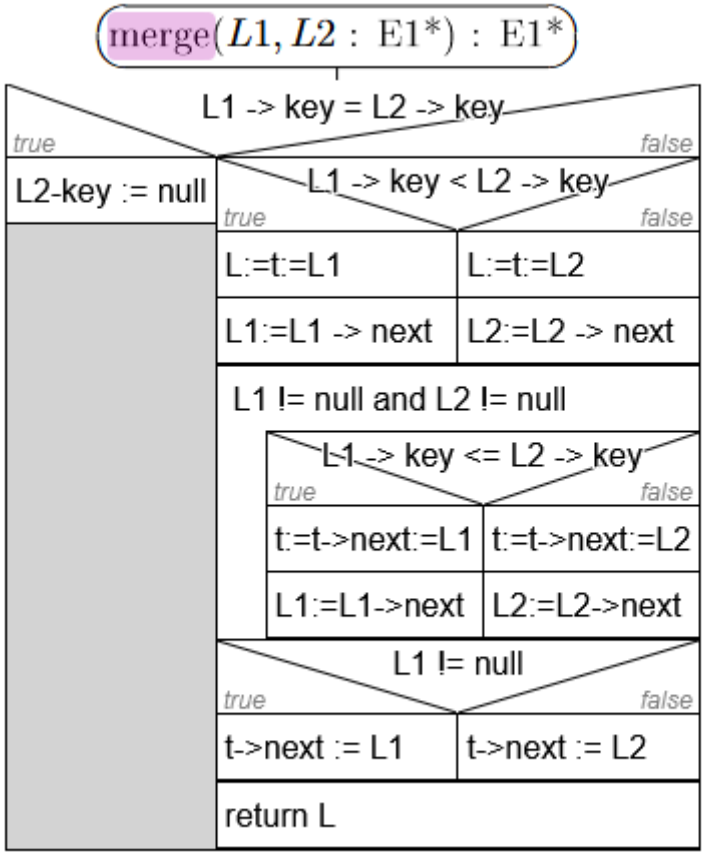
Tárigény:

Tömbökre: $\Theta(n)$

Láncolt listákra: $\Theta(\log n)$

5.a mergesort

5.b



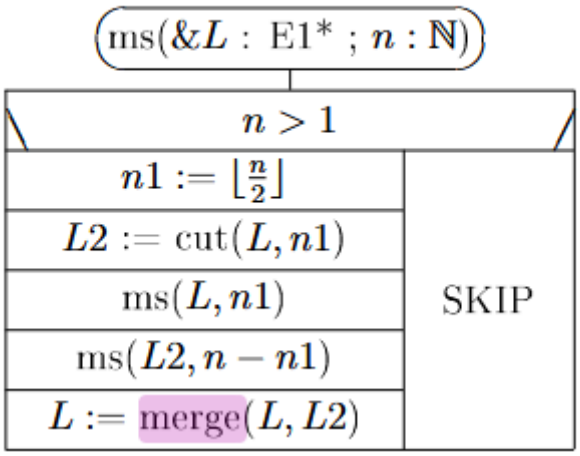
5.c

Ha L1 hossza n és L2 hossza m akkor a merge műveletigénye:
$\Theta(n+m)$

Tárigénye:
$\Theta(n+m+2)$ *(nem biztos, hogy jó)*

6.a mergesort

6.b



6.c log n+1-szer, mert a ms(L,n) eljárás minden hívásban felezi az rendezendő részlista hosszát

2.3 Kupacrendezés

1.a

szigorúan bináris fa: a fa minden belső pontjának két gyereke van.

teljes bináris fa: olyan szigorúan bináris fa, ahol minden levél azonos szinten helyezkedik el.

majdnem teljes bináris fa: olyan teljes bináris fa, melynek legalsó (levél) szintjéről elhagytunk néhány levelet (de nem az összeset).

majdnem teljes balra tömörített bináris fa: majdnem teljes bináris fa, de levelek csak a legalsó szintről, a jobb oldalról hiányozhatnak. Ezeket szintfolytonos fáknak is nevezzük.

kupac: maximum vagy minimum kupac lehet. Maximum kupac: egy majdnem teljes, balra tömörített bináris fa, melynek minden belső pontjára teljesül, hogy a belső pont kulcsa nagyobb vagy egyenlő a gyerekei kulcsánál.

Így kupac tetején (a fa gyökerében) mindig az egyik legnagyobb elem található.

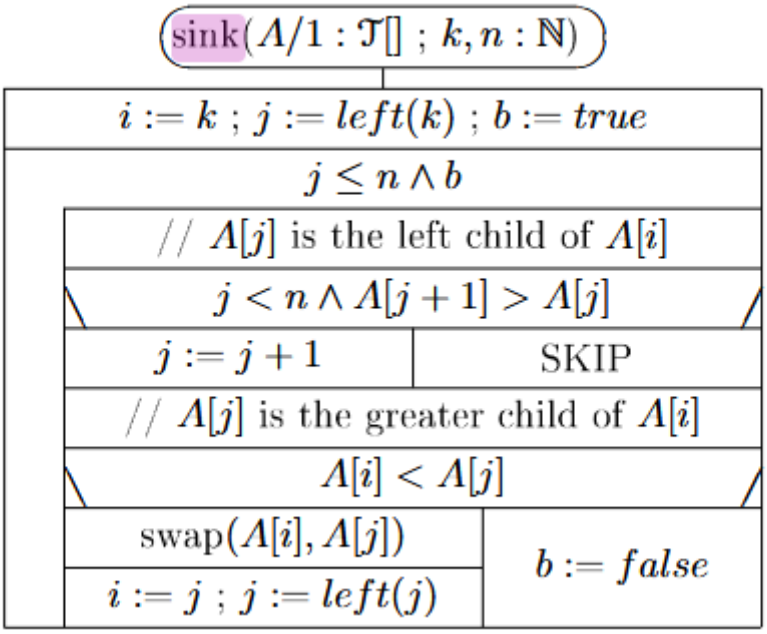
Minimum kupac hasonlóan: a szülő kulcs kisebb vagy egyenlő a gyerekei kulcsánál.

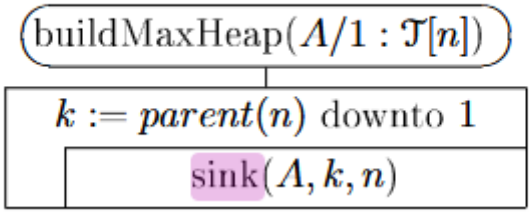
1.b kupacrendezés

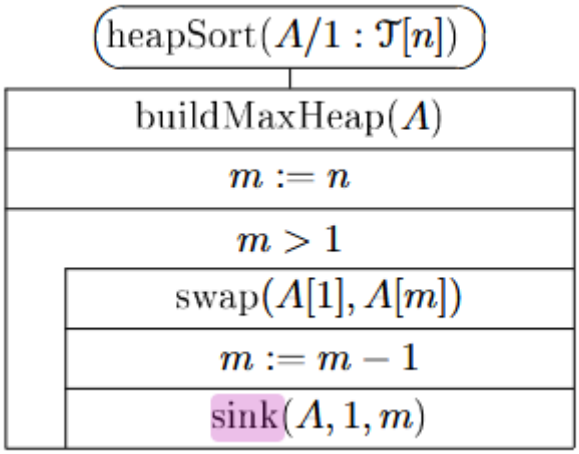
2.a -> 1.a-nál megválaszolva

2.b kupacrendezés

3.a


sink($A/1 : \mathcal{T}[]$; $k, n : \mathbb{N}$)
$i := k$; $j := left(k)$; $b := true$
$j \leq n \wedge b$
// $A[j]$ is the left child of $A[i]$
$j < n \wedge A[j+1] > A[j]$
$j := j + 1$
SKIP
// $A[j]$ is the greater child of $A[i]$
$A[i] < A[j]$
swap($A[i], A[j]$)
$i := j$; $j := left(j)$
$b := false$


buildMaxHeap($A/1 : \mathcal{T}[n]$)
$k := parent(n)$ downto 1
sink(A, k, n)


heapSort($A/1 : \mathcal{T}[n]$)
buildMaxHeap(A)
$m := n$
$m > 1$
swap($A[1], A[m]$)
$m := m - 1$
sink($A, 1, m$)

3.b

9.10.1. A kupacrendezés műveletigénye

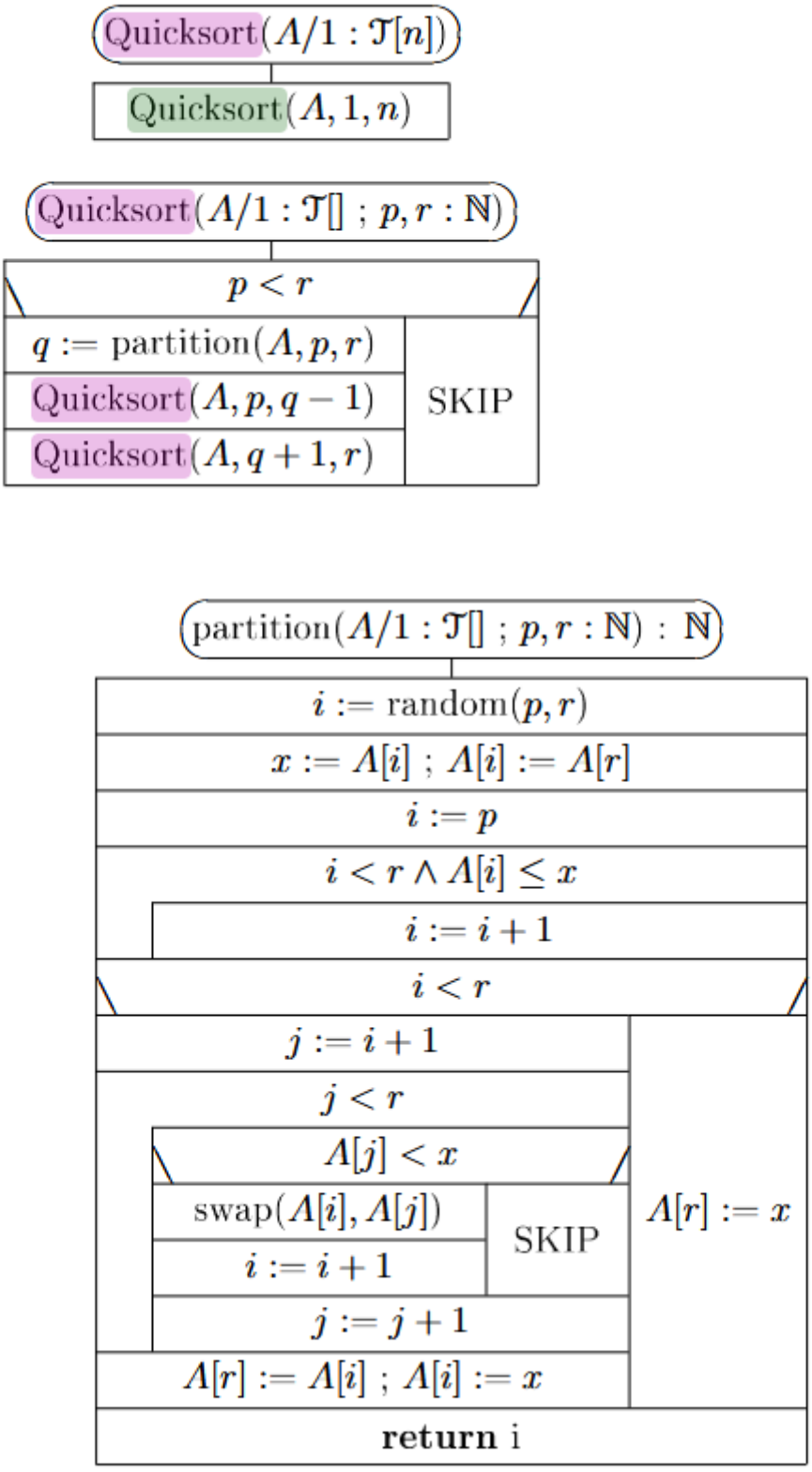
A kupaccá alakítás utáni rész futási idejét a sink$(A, 1, m)$ hívások határozzák meg, amiknek műveletigénye $O(\log m)$ $(m \in \{n-1, n-2, \ldots, 1\})$, durva felső becsléssel $O(\log n)$. Az $(n-1)$ hívás tehát összesen $O(n \log n)$ futási idejű, és ezt nagyságrendileg a ciklus $(n-1)$ iterációja nem befolyásolja, mert ez csak $O(n)$ műveletigényt ad hozzá, ami aszimptotikusan kisebb, mint $O(n \log n)$.

A kupaccá alakítás műveletigényéről hasonlóan látható, hogy maga is $O(n \log n)$, ui. az iteráció és a sink(A, k, n) eljáráshívás is $\lfloor \frac{n}{2} \rfloor$-ször hajtódik végre, és a k gyökerű részfa magassága $\leq \log n$, így az eljáráshívás műveletigénye durva felső becsléssel $O(\log n)$. Az $\lfloor \frac{n}{2} \rfloor$ hívás tehát összesen $O(n \log n)$ futási idejű, és ezt nagyságrendileg a ciklus $\lfloor \frac{n}{2} \rfloor$ iterációja nem befolyásolja, mert ez csak $O(n)$ műveletigényt ad hozzá, ami aszimptotikusan kisebb, mint $O(n \log n)$.

10.4. Tétel. *Tetszőleges összehasonlító rendezésre* $MT(n) \in \Omega(n \log n)$.

Ezek miatt az állításunk igaz.

2.4 Gyorsrendezés



1.b

Az előadásjegyzeten a 36-37-38.oldal. A tömbös rajz szerűség.

1.c

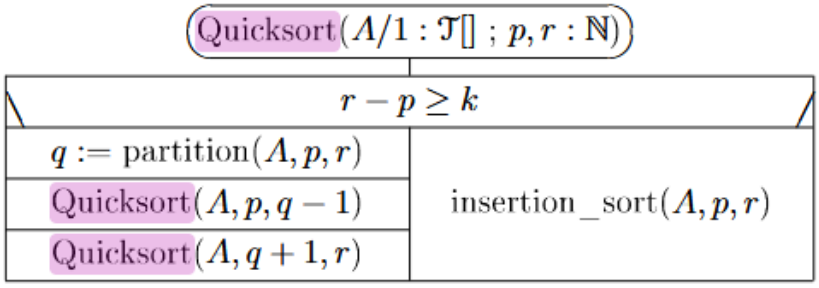
mT(n), AT(n)∈ $\Theta(n * \log n)$

MT(n) ∈ $\Theta(n^2)$

1.d

Igen, mert ha kis méretű résztömböknél áttérünk beszúró rendezésre, akkor a Quicksort(A,p,r) eljárás jelentősen gyorsítható.

Ennek stuktogrammos ábrázolása.



Itt $k \in \mathbb{N}$ konstans. Optimális értéke sok tényezőtől függ, de általában 20 és 40 között mozog.

2.a -> 1.a-nál megválaszolva

2.b ->1.b-nél leírva

2.c -> 1.c-nél leírva

2.d

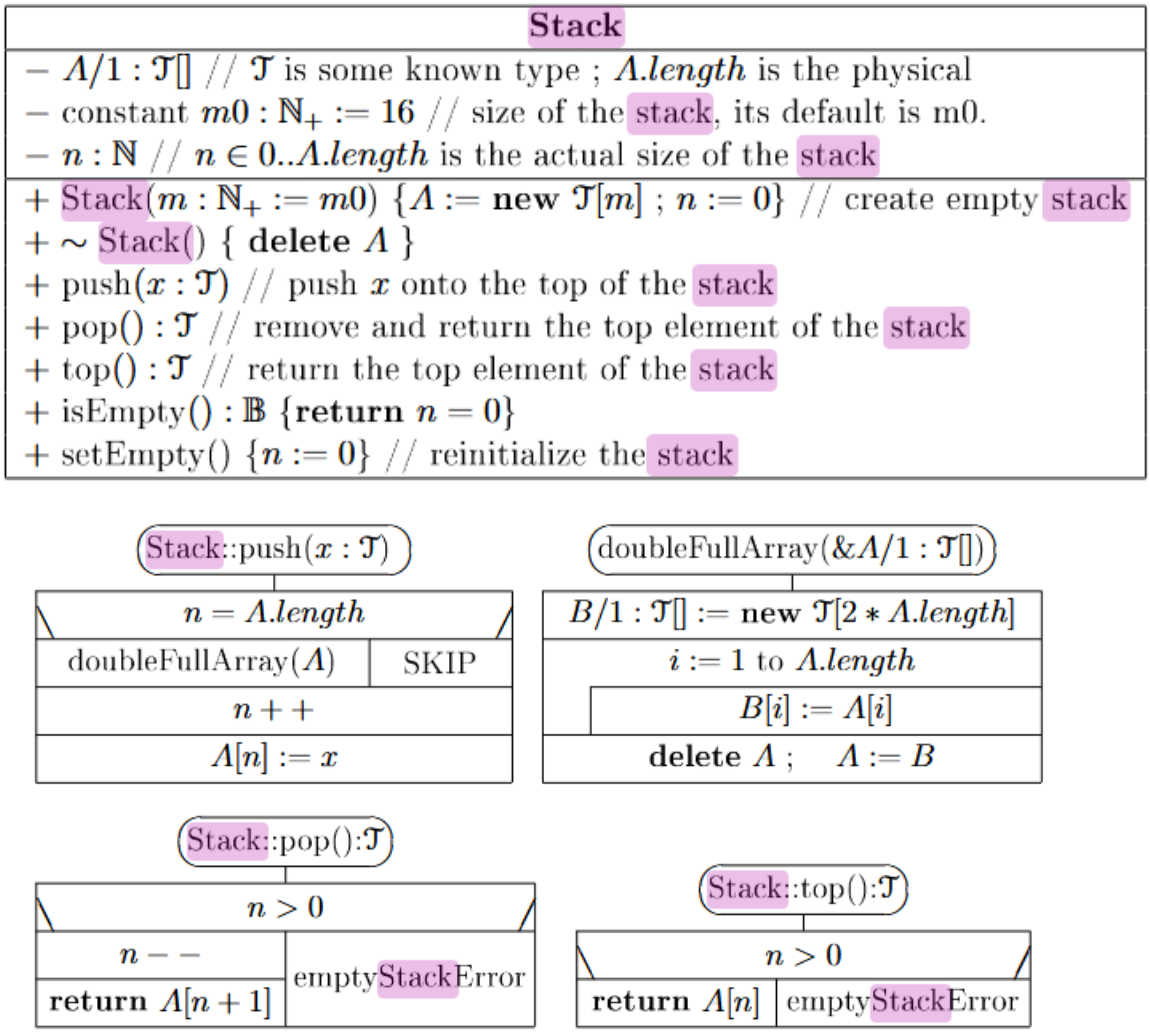
legrosszabb esetben:
$\Theta(\log n)$
$\Theta(n^2)$
Hogy miért azt nem tudom ☹.

3. –

3. Absztrakt adattípusok

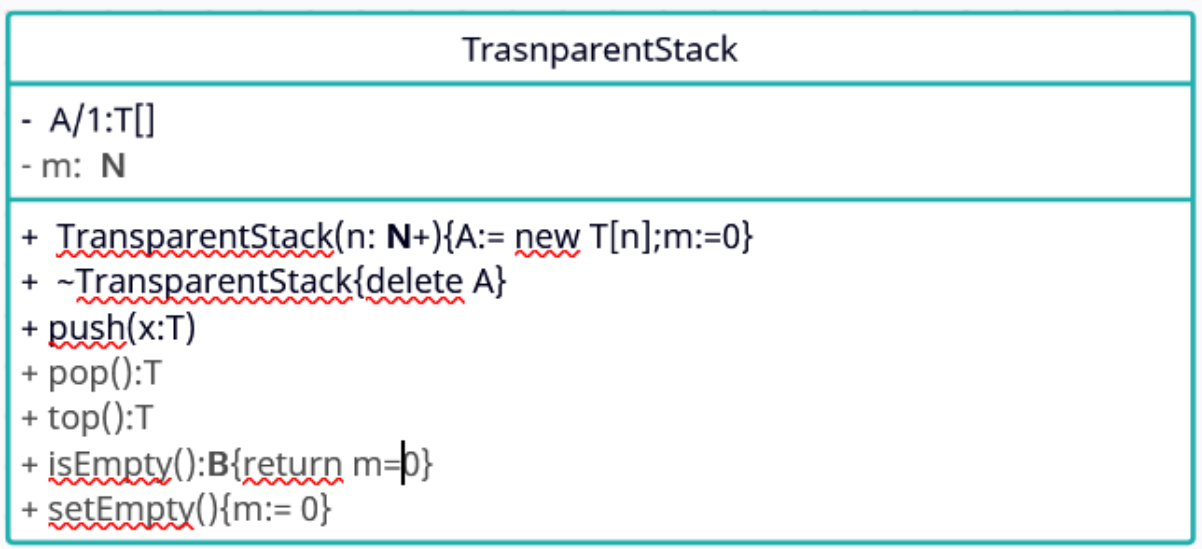
3.1 Verem

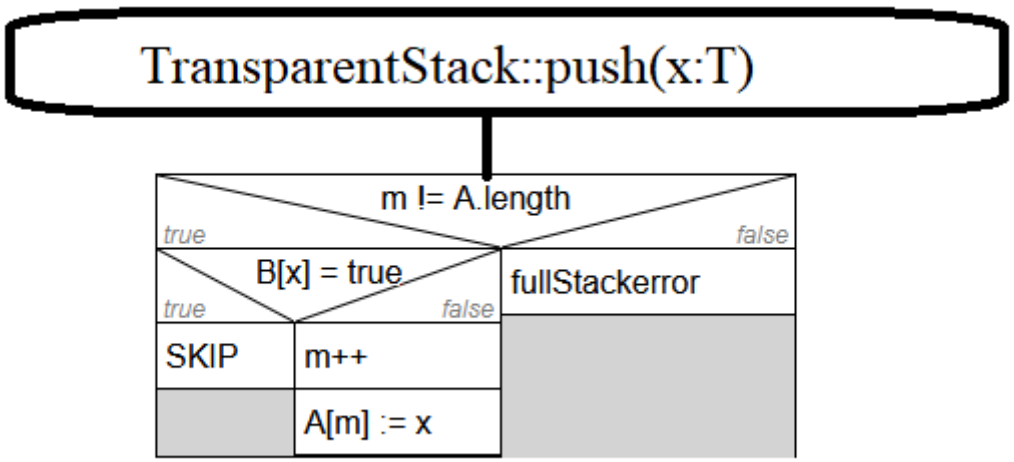
1.



Mindegyiké $\Theta(1)$ kivéve a push-é, mert az nyílván mT(n) ∈ $\Theta(1)$ és MT(n) ∈ $\Theta(n)$

2.





A többi metódus ugyanaz csak n helyett m és Stack:: helyett TransparentStack::

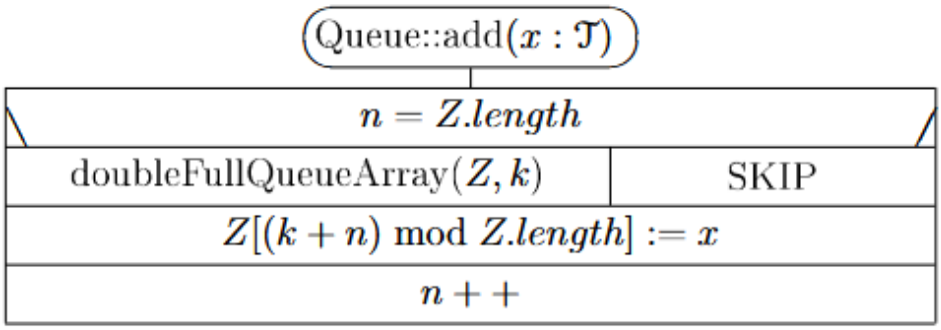
3. –

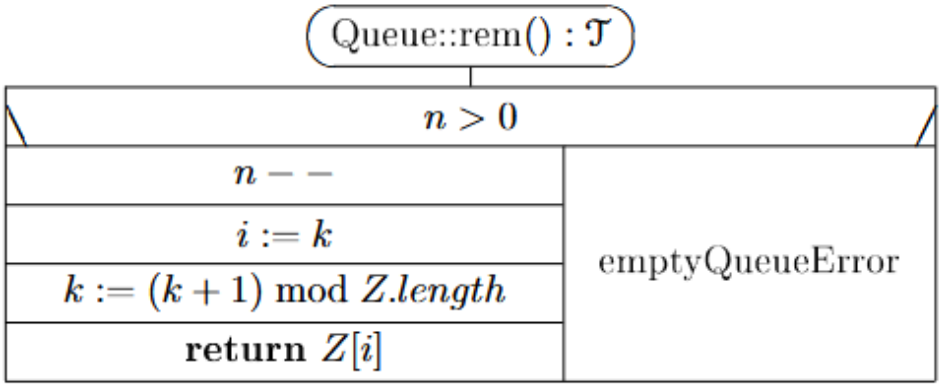
3.2 Sor

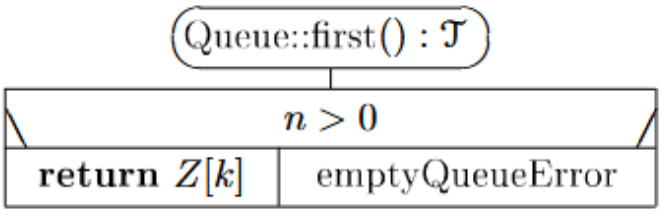
1.

Queue
$-\ Z : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $Z.length$ is the physical $-$ constant $m0 : \mathbb{N}_+ := 16$ // length of the queue, its default is m0. $-\ n : \mathbb{N}$ // $n \in 0..Z.length$ is the actual length of the queue $-\ k : \mathbb{N}$ // $k \in 0..(Z.length - 1)$: the starting position of the queue in Z

+ Queue($m : \mathbb{N}_+ := m0$){ $Z :=$ **new** $\mathcal{T}[m]$; $n := 0$; $k := 0$ }
// create an empty queue
+ add($x : \mathcal{T}$) // join x to the end of the queue
+ rem() : $\mathcal{T}$ // remove and return the first element of the queue
+ first() : $\mathcal{T}$ // return the first element of the queue
+ length() : $\mathbb{N}$ {**return** n}
+ isEmpty() : $\mathbb{B}$ {**return** $n = 0$}
+ $\sim$ Queue() { **delete** Z }
+ setEmpty() {$n := 0$} // reinitialize the queue







a doubleFullQueueArray ugyanaz, mint a stacknél a doubleFullArray

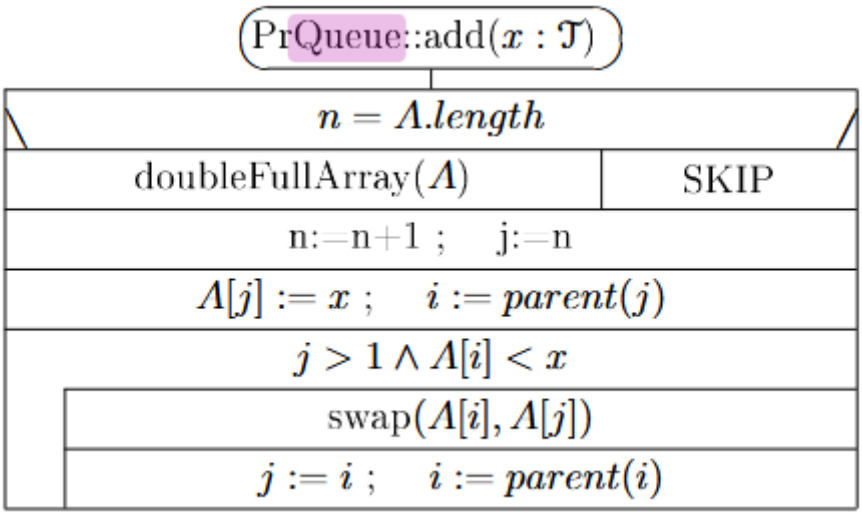
2.-

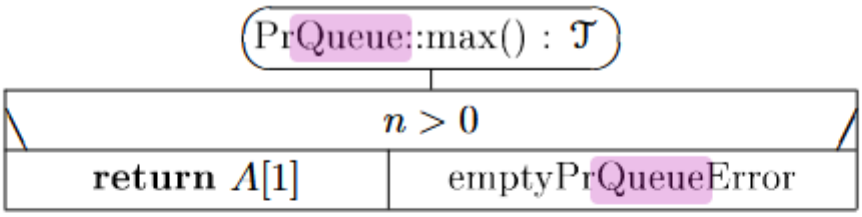
3.-
(*Ezekhez már nem volt agyi kapacitásom*)

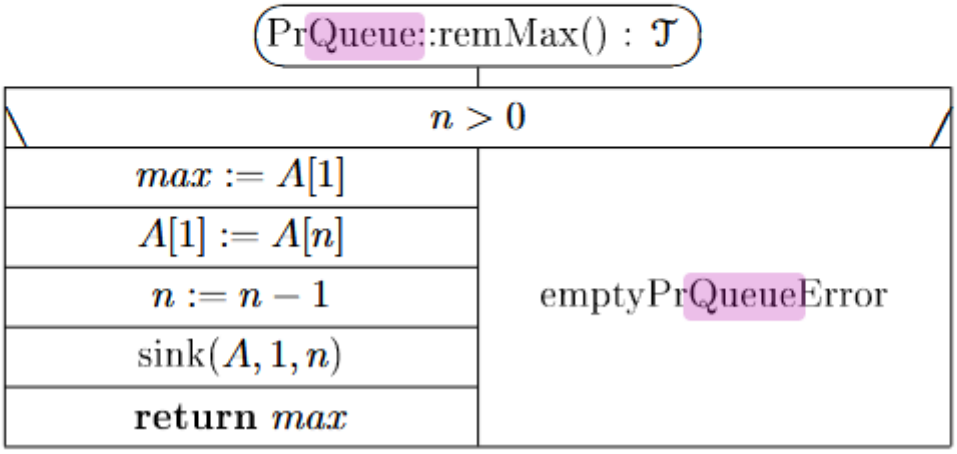
3.3 Prioritásos sor

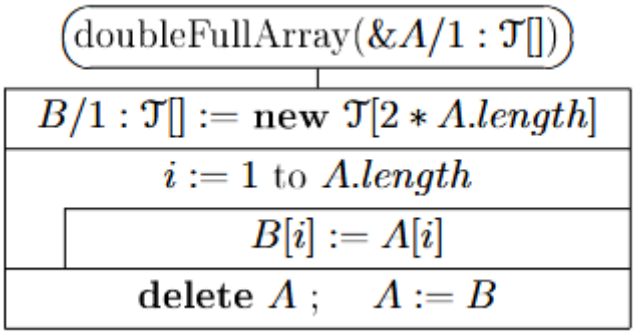
1.

PrQueue
− $A/1 : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $A.length$ is the physical − constant $m0 : \mathbb{N}_+ := 16$ // size of the PrQ, its default is m0. − $n : \mathbb{N}$ // $n \in [0..A.length]$ is the actual length of the PrQ
+ PrQueue$(m : \mathbb{N}_+ := m0)\{\ A := \textbf{new}\ \mathcal{T}[m];\ n := 0\ \}$ // create an empty PrQ + add$(x : \mathcal{T})$ // insert x into the priority queue + remMax():$\mathcal{T}$ // remove and return the maximal element of the priority queue + max():$\mathcal{T}$ // return the maximal element of the priority queue + isEmpty() : $\mathbb{B}$ $\{\textbf{return}\ n = 0\}$ + $\sim$ PrQueue() $\{\ \textbf{delete}\ A\ \}$ + setEmpty() $\{n := 0\}$ // reinitialize the priority queue









Ha az A tömb n hosszúságú kezdőszelete egy kupac aritmetikai ábrázolása,
$MT_{\text{add}:n<A.length}(n) \in \Theta(\log n)$, $MT_{\text{add}:n=A.length}(n) \in \Theta(n)$,
$AT_{\text{add}}(n) \in O(\log n)$, $mT_{\text{add}}(n) \in \Theta(1)$,
$MT_{\text{remMax}}(n) \in \Theta(\log n)$, $mT_{\text{remMax}}(n) \in \Theta(1)$, $T_{\max}(n) \in \Theta(1)$.

2.a -> A kupacrendezéses kérdéseknél az 1.a

2.b kupacbeszúrás (algo gyak Gyak09)

2.c kupac maximumkivétel (algo gyak Gyak09)

3.

Én ide a sima PrQueue-t rajzoltam, de nem tudom jó-e
és a műveletigény:

$MT_{\mathbf{add}:n<A.length}(n) \in \Theta(\log n)$, hiszen a ciklus legfeljebb annyiszor iterál, amennyi a fa magassága, azaz (n input értékéhez viszonyítva) $\lfloor \log(n+1) \rfloor$-szer, amiből $\log n \leq MT_{\mathbf{add}:n<A.length}(n) = \lfloor \log(n+1) \rfloor + 1 \leq \log n + 2$.

$MT_{\mathbf{add}:n=A.length}(n) \in \Theta(n)$, hiszen a doubleFullArray(A) eljáráshívás ciklusa $n = A.length$ iterációt hajt végre, ami dominál az x fölemelése és egyéb tevékenységek $O(\log n)$ műveletigénye fölött.

$mT_{\mathbf{add}}(n) \in \Theta(1)$, ha ugyanis $n < A.length$, akkor a doubleFullArray(A) eljárás nem hívódik meg, és ha a kupachoz adott elem (x) elég kicsi, akkor a ciklus egyet sem iterál, innét pedig $mT_{\mathbf{add}}(n) = 1$.

$T_{\mathbf{max}}(n) \in \Theta(1)$, mivel $T_{\mathbf{max}}(n) = 1$.

Innét a kupacnak a remMax meghívásának pillanatában érvényes n méretére
$\log n \leq \log(n-1) + 1 \leq MT_{\mathbf{remMax}}(n) \leq \log(n-1) + 2 \leq \log n + 2$.
Ebből $MT_{\mathbf{remMax}}(n) \in \Theta(\log n)$.

$mT_{\mathbf{remMax}}(n) \in \Theta(1)$, hiszen
$2 = 1 + 1 \leq mT_{\mathbf{remMax}}(n) = 1 + mT_{\mathbf{sink}}(n-1) \leq 1 + 2 = 3$.

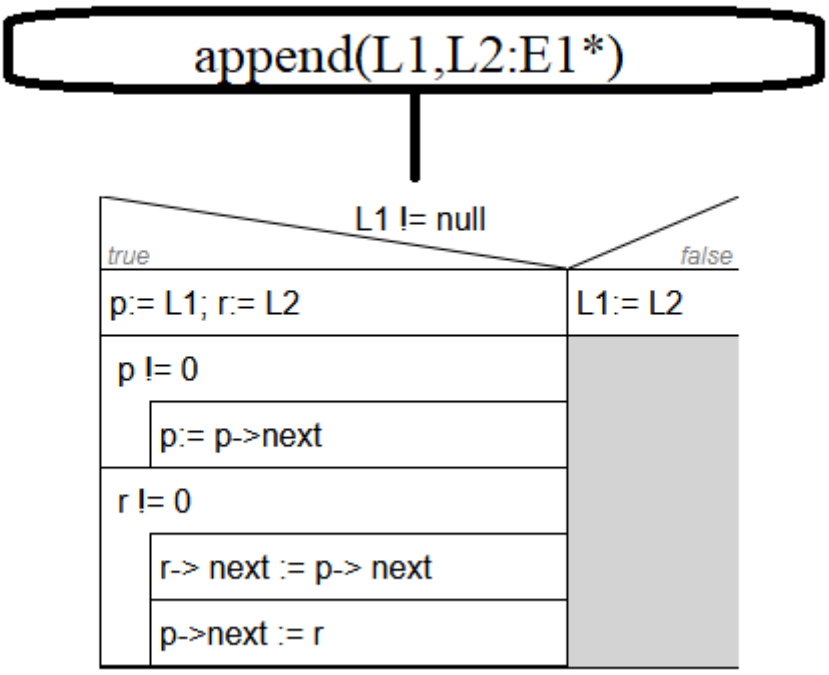
4. Itt nem tudom mit akart Ásványi ☹

5. 6 – Sajnos a fejelemes simáról nem találtam semmit, a csökkenőset meg nem tudom, hogy kellett.

4. Láncolt listák

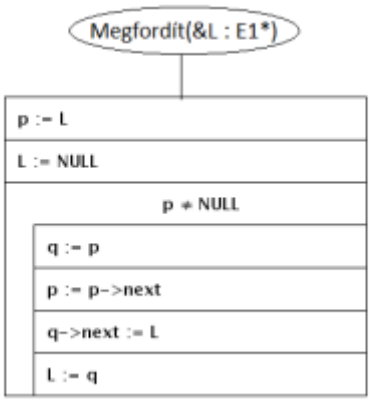
4.1 Egyszerű listák(S1L)

1.



Megj.: Saját készítés, ezért nem biztos, hogy jó.

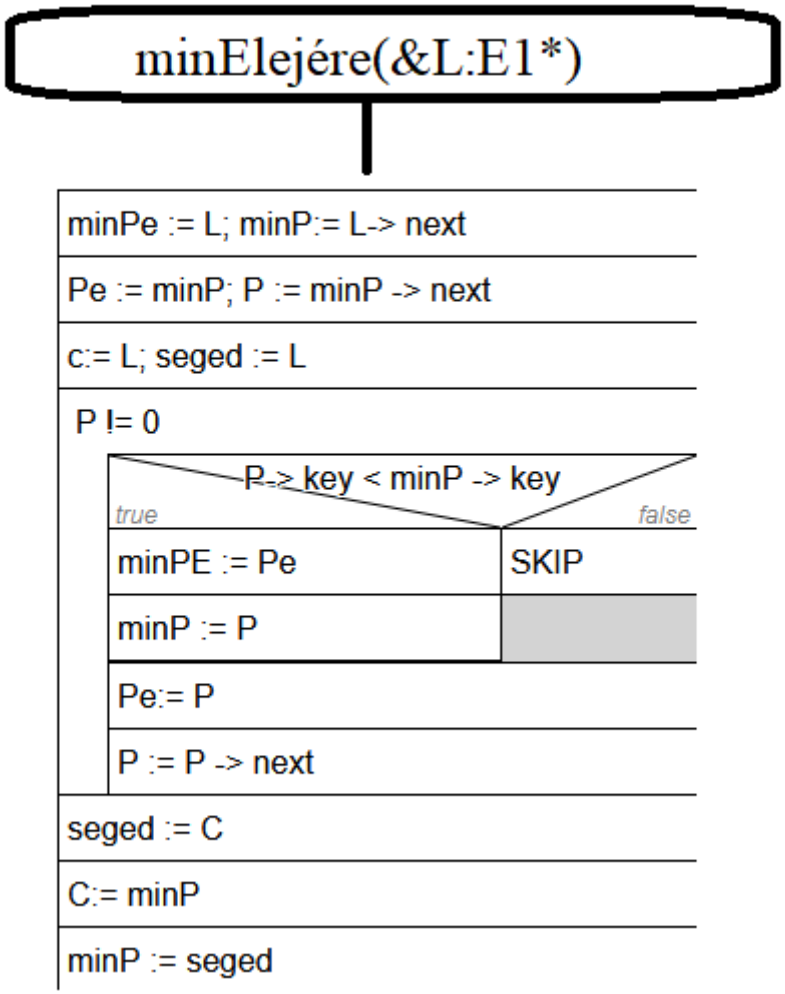
2.



4.2 Fejelemes listák(S1L)

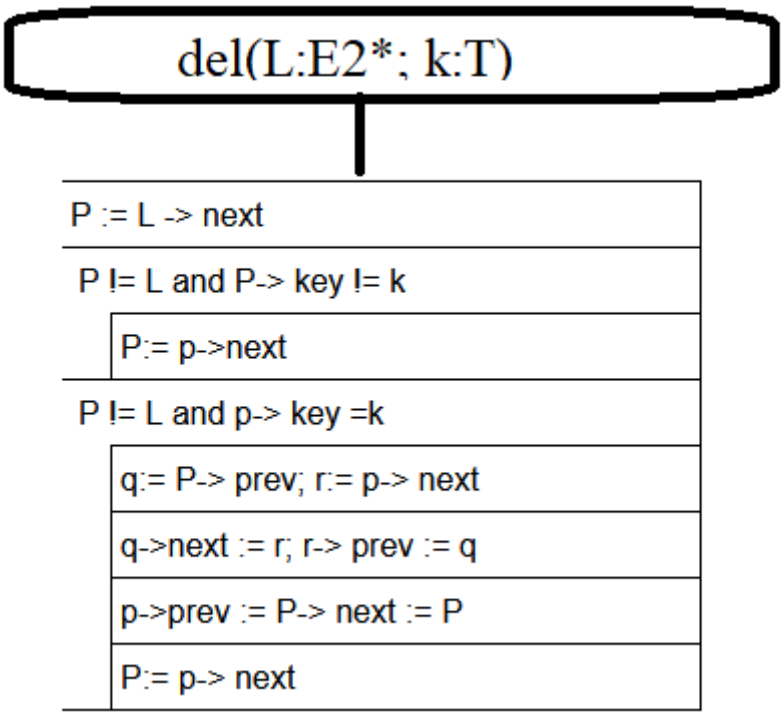
1. -

2. -



4.3 Egyszerű Kétirányú listák(S2L)

1.

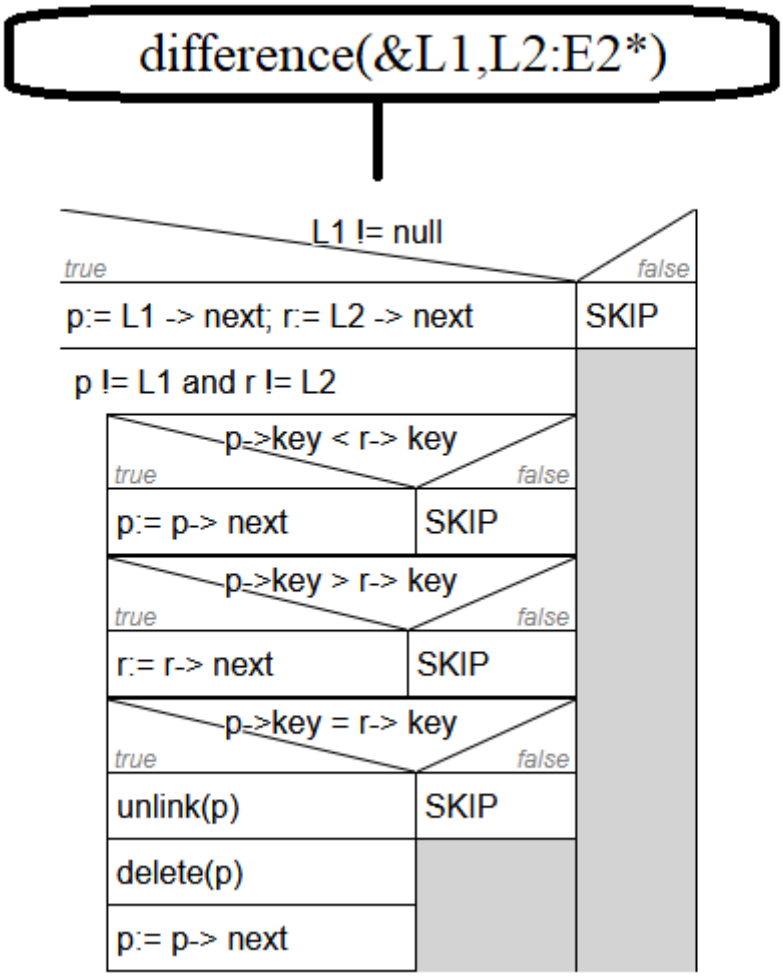


MT(n) ∈ $\Theta(n)$, mert ha mindegyik elem a listában van akkor mindegyiket törölni kell
mT(n) ∈ $\Theta(n)$, mert ha nincs benne az elem akkor is legalább egyszer végig kell mennünk az első cikluson. Itt n a lista hossza

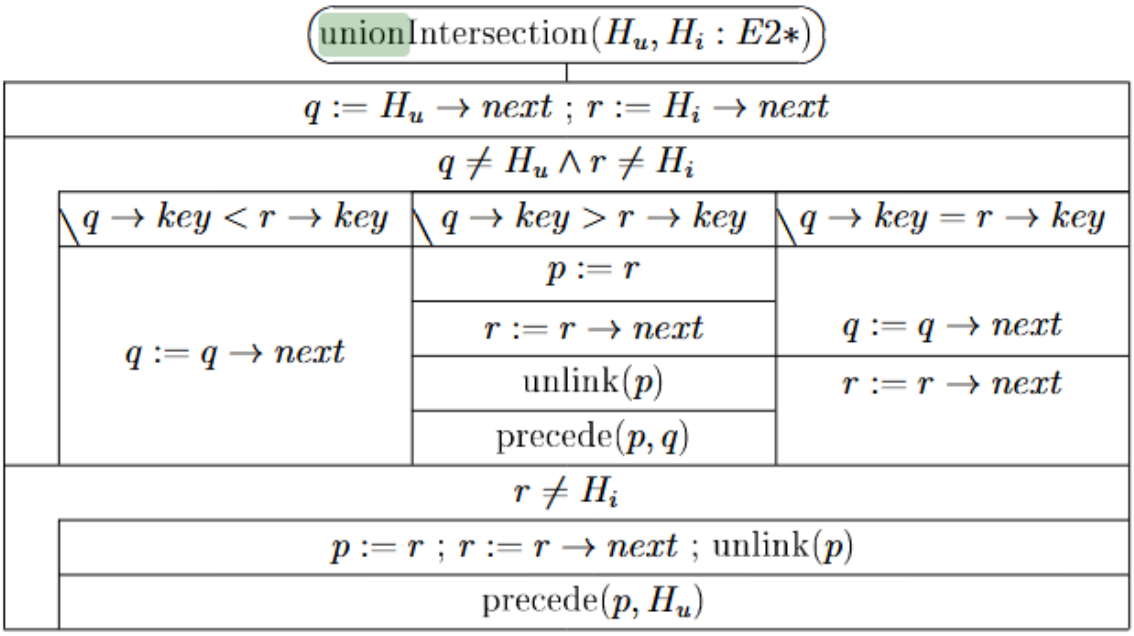
2.-

4.4 Fejelemes, Kétirányú, ciklikus listák(C2L)

1.


difference(&L1,L2:E2*)
L1 != null
true
false
p:= L1 -> next; r:= L2 -> next
SKIP
p != L1 and r != L2
p->key < r-> key
true
false
p:= p-> next
SKIP
p->key > r-> key
true
false
r:= r-> next
SKIP
p->key = r-> key
true
false
unlink(p)
SKIP
delete(p)
p:= p-> next

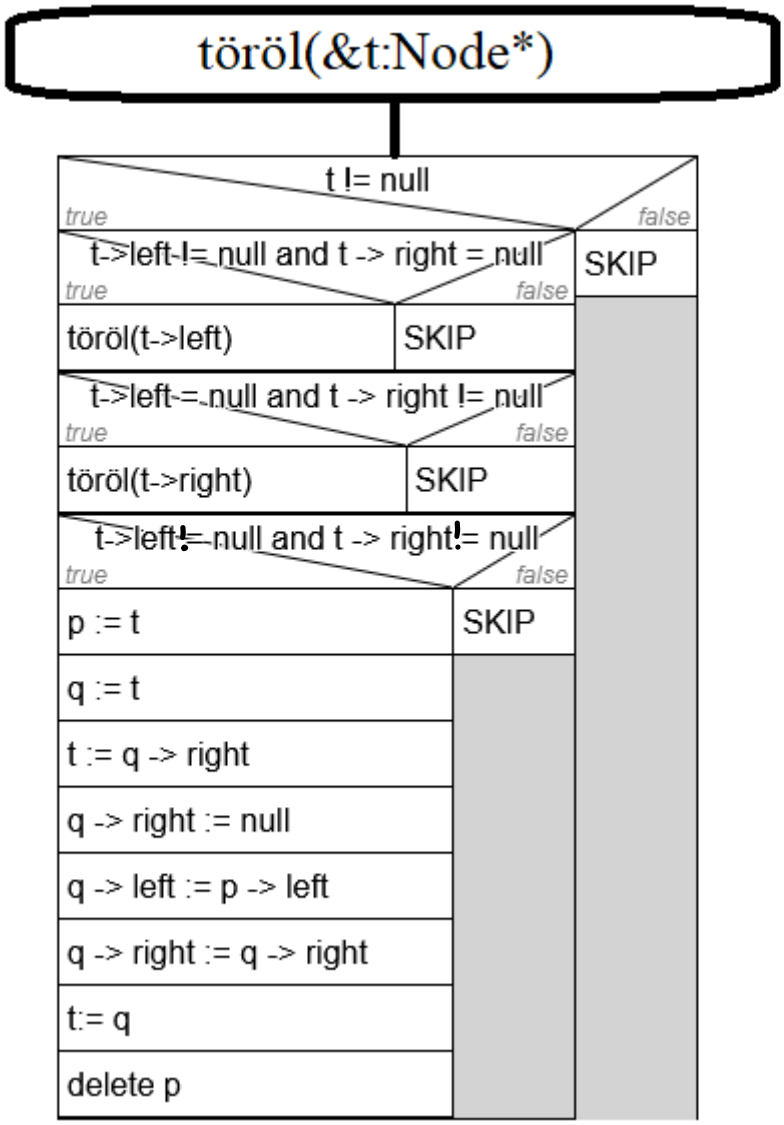
2.



u Helyére 1-et és i helyére 2-t írj!

5. Bináris fák

1.a



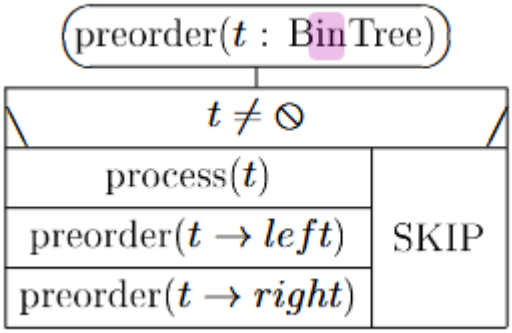
1.b –

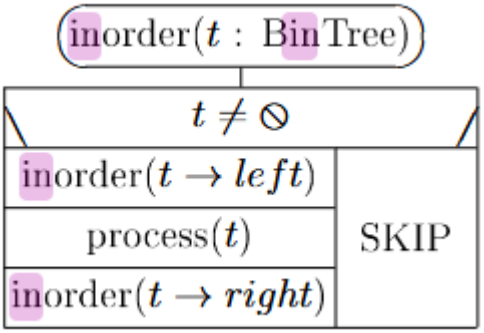
2. –

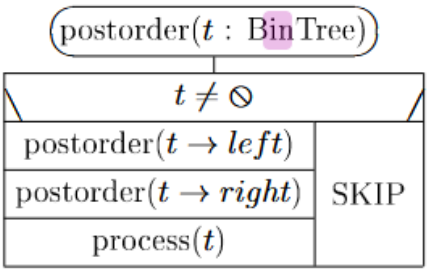
3. –

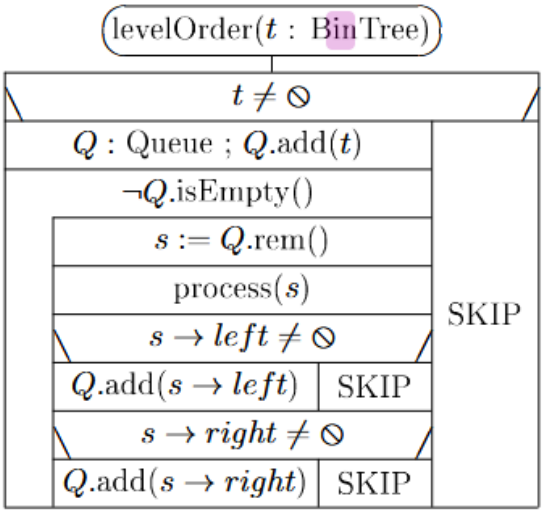
4. preorder, inorder, postorder, levelOrder(szintfolytonos)

4.a









4.b

Állítás: $T_{\text{preorder}}(n), T_{\text{inorder}}(n), T_{\text{postorder}}(n), T_{\text{levelOrder}}(n) \in \Theta(n)$, ahol n a fa *mérete*, azaz csúcsainak száma.

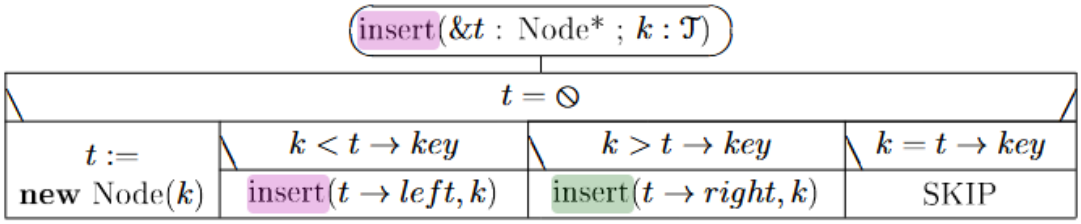
Ez azért igaz, mert egy- egy hívás végrehajtásához $\Theta(1)$ futási idő kell.

5.1 Bináris keresőfák

1.a

Tetszőleges csúcs kulcsánál a bal részfájában minden csúcs kulcsa kisebb, a jobb részfájában minden csúcs kulcsa nagyobb.

1.b



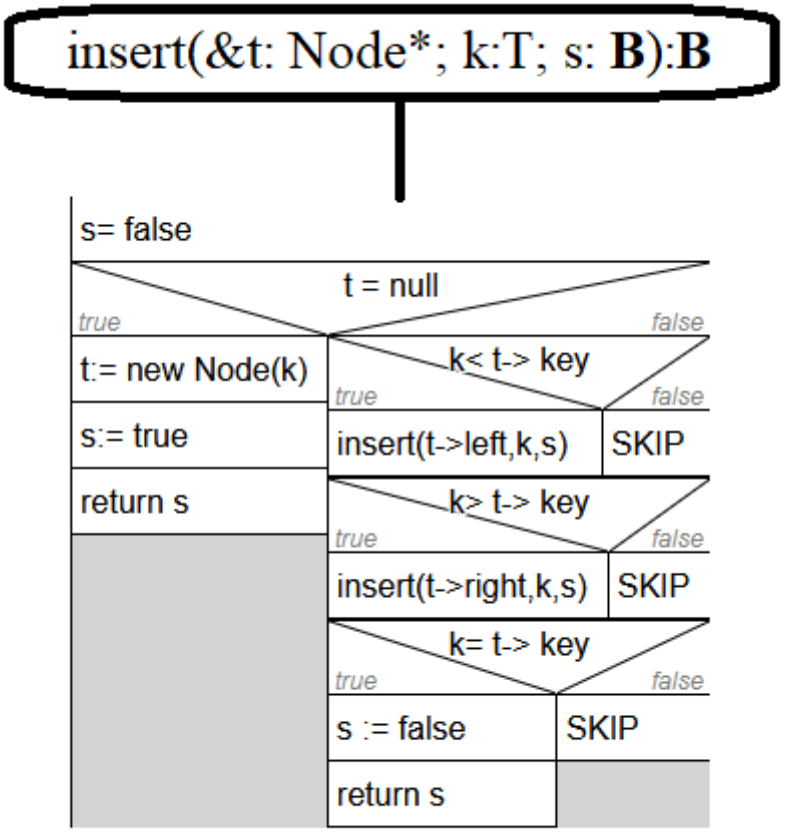
1.c

mT(n) ∈ $\Theta(1)$, mert ha a fa üres a kezdeténél. akkor csak beszúrunk egy új Node-ot.

MT(h) ∈ $\Theta(h)$, ez a bináris keresőfa magasságától(h) függ. Ha a fának n csúcsa van, akkor a magassága $\lfloor \log n \rfloor$ és (n-1) között változik

2.a -> 1.a-nál megválaszolva

2.b



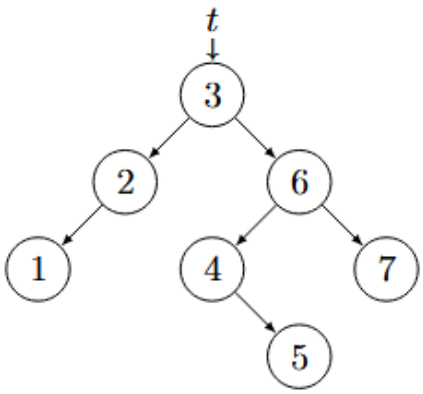
2.c

Igaz, mert sok alkalmazás esetén azonban nem megengedhető az a kockázat, hogy ha a keresőfa(majdnem) listává torzul, akkor a műveletek hatékonysága is hasonló lesz, mint a láncolt listák esetén. Az ideális megoldás az lenne, ha tudnánk garantálni, hogy a fa majdnem teljes legyen.

Nem ismerünk azonban olyan $O(\log n)$ futási idejű algoritmusokat, amelyek pl. a beszúrás és törlési műveletek során ezt biztosítani tudnák.

3.a -> 1.a-nál megválaszolva

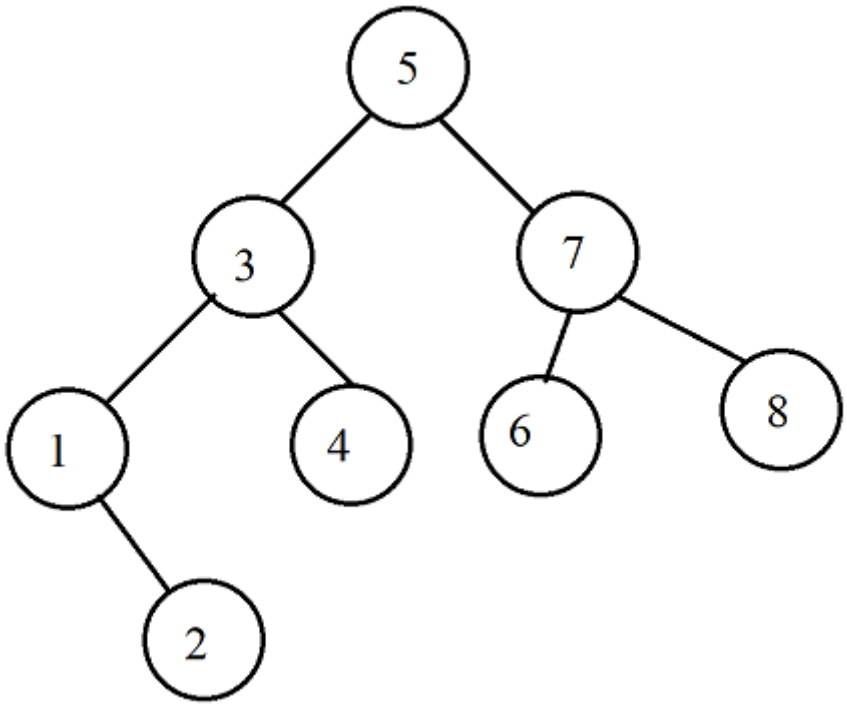
3.b



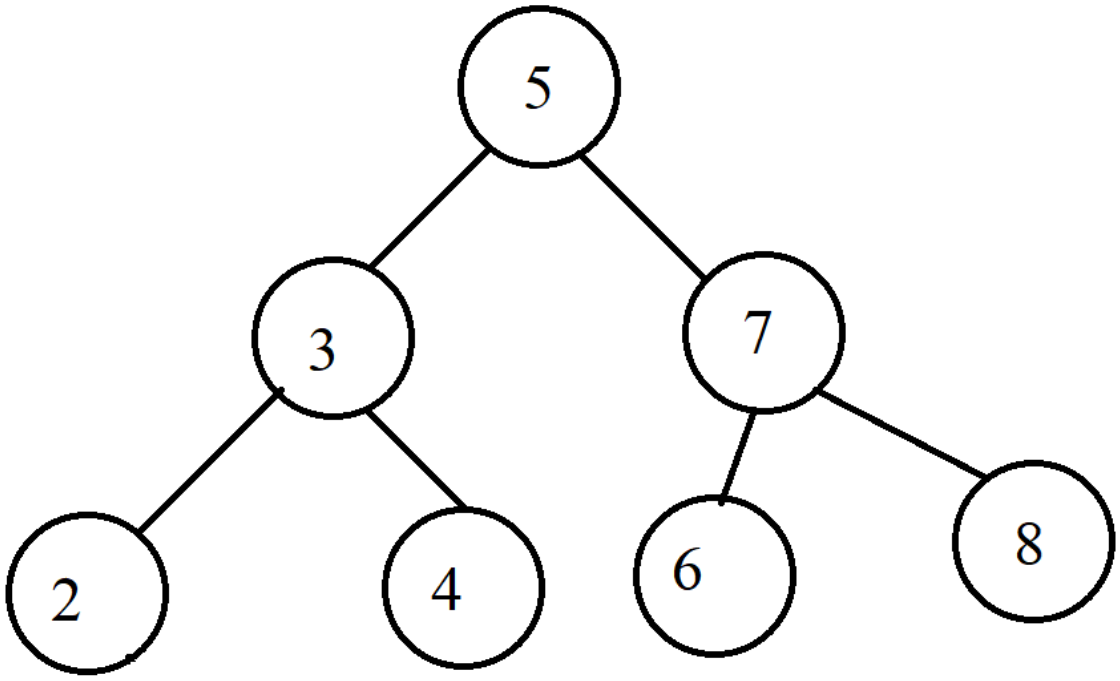
A balra látható t bináris fa

- egyszerű zárójeles alakja:
  (((1) 2) 3 ((4 (5)) 6 (7)))
- elegáns zárójeles alakja:
  { [(1) 2] 3 [(4 ⟨5⟩) 6 (7)] }

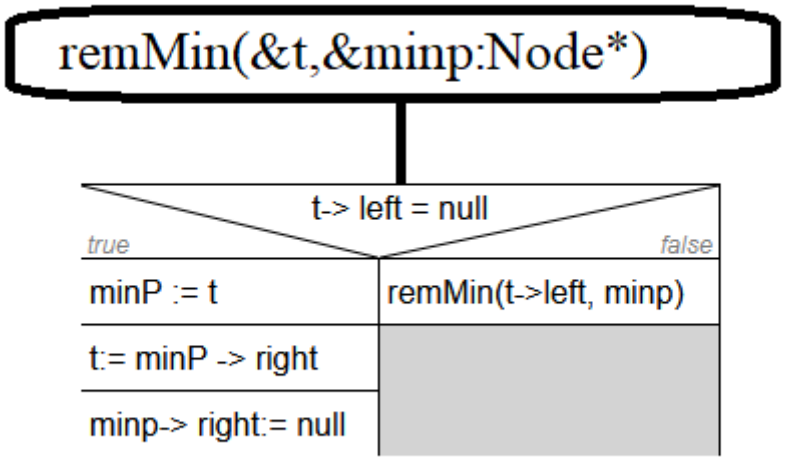
eszerint a feladat fája:

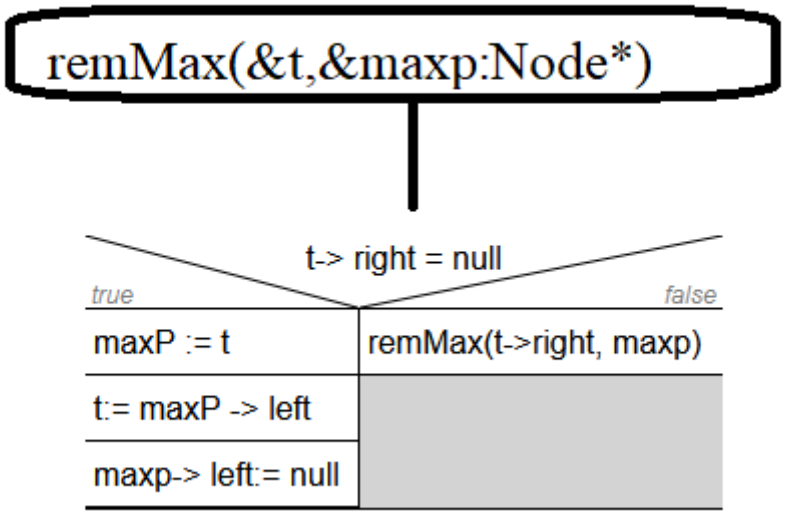


Ha kivesszük a legkisebbet akkor ezt a fát kapjuk:


5
3
7
2
4
6
8

3.c


remMin(&t,&minp:Node*)
t-> left = null
true
false
minP := t
remMin(t->left, minp)
t:= minP -> right
minp-> right:= null

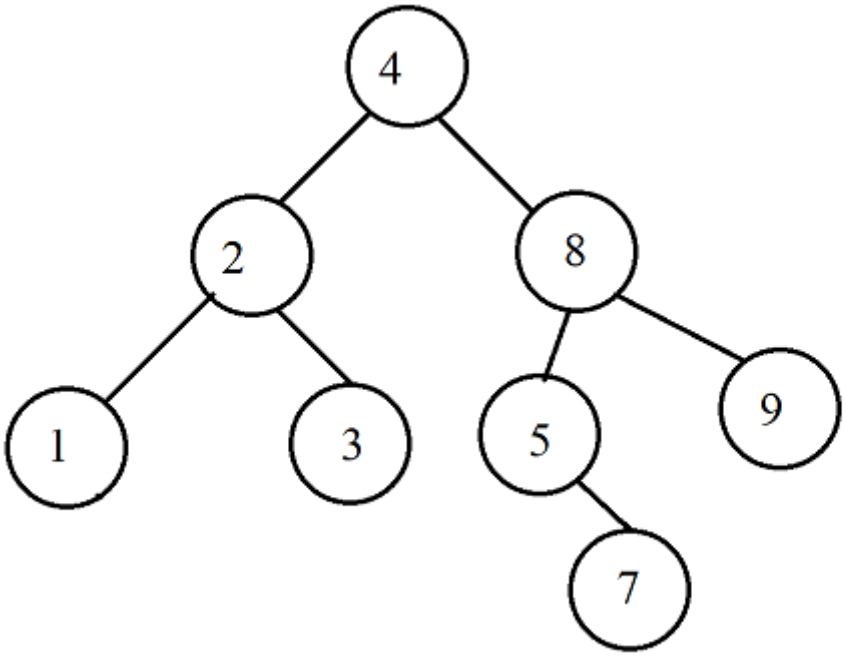


3.d -> 2.c -nél

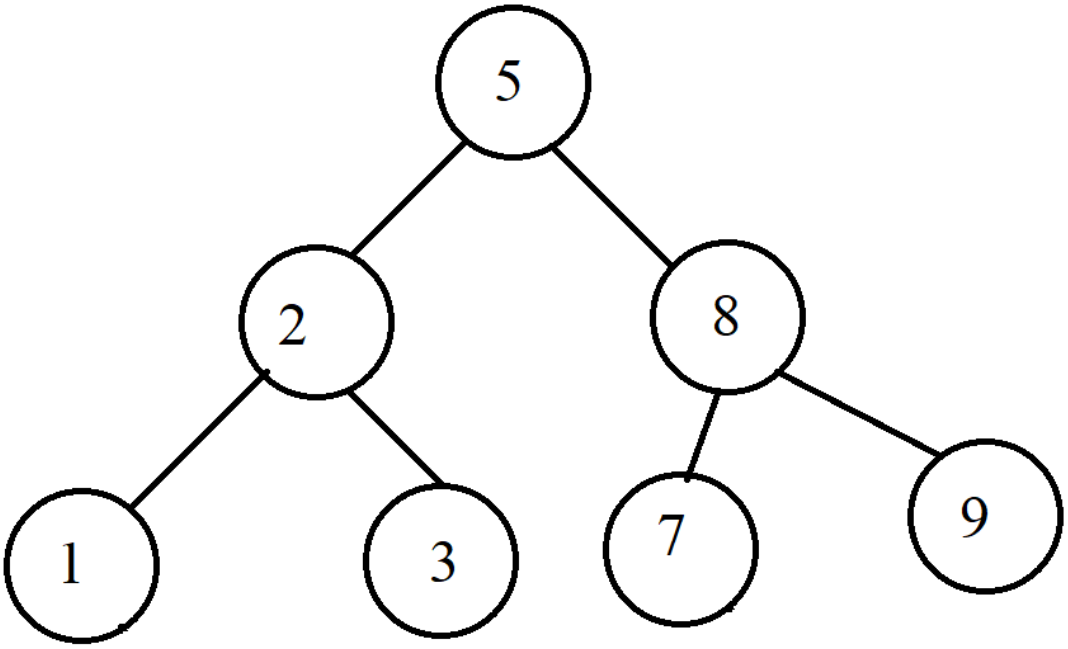
4.a -> 1.a-nál

4.b

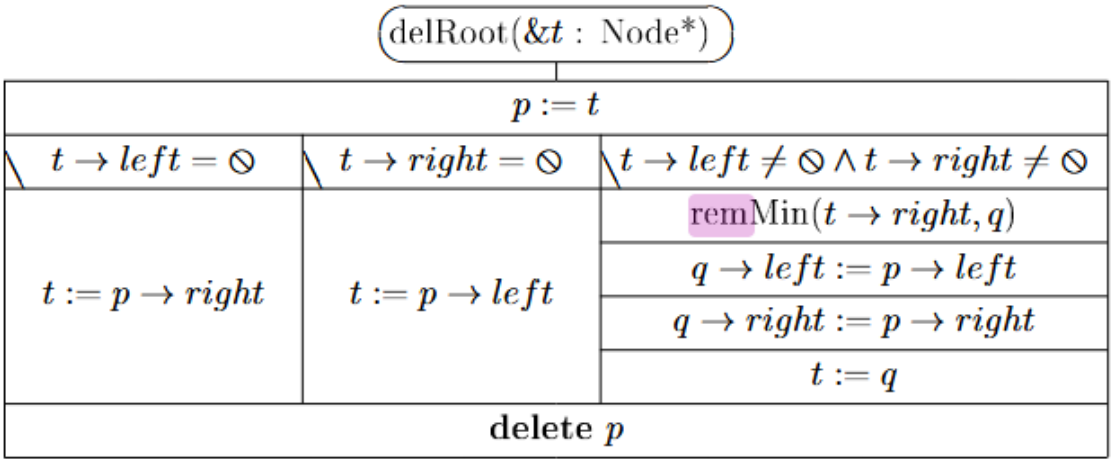
A feladat fája:



Kitöröljük a 4-et és a helyére az 5 kerül. aminek helyére a 7 kerül. A delRoot után a kereső fa így néz ki:



4.c

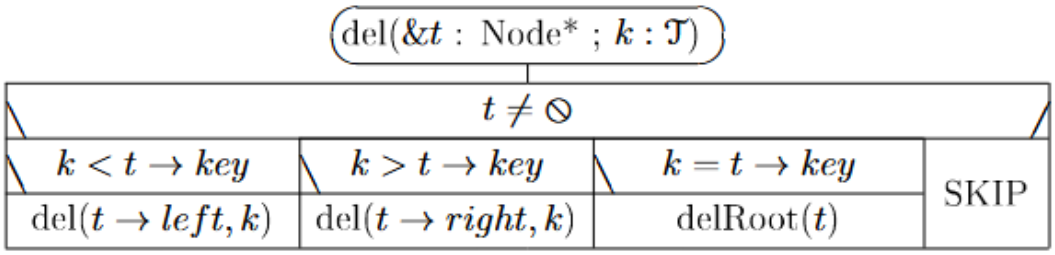


4.d -> 2.c

5.a -> 1.a

5.b > 4.b-hez hasonlóan

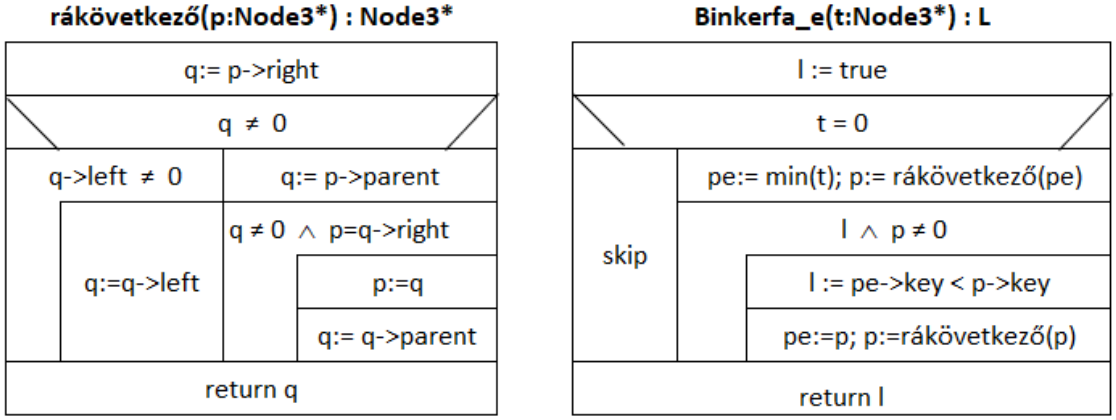
5.c



5.d -> 2.c

6.a -> 1.a

6.b



6.c

Igen, mert legjobb esetben csak „felfele” kell mennünk, és ha a gyökérnek nincs jobb részfája akkor igaz.

7.a

Az $n - 1 \geq h$ egyenlőtlenséghez gondoljuk meg, hogy tetszőleges h magasságú fa szintjeit 0-tól h-ig sorszámoztuk, ami összesen h+1 szintet jelent. Mivel a fa minden szintjén van legalább egy csúcs, ezért tetszőleges fára igaz, hogy $h \leq n - 1$.

7.b

h = n-1 pontosan akkor teljesül, ha a fa listává torzult.

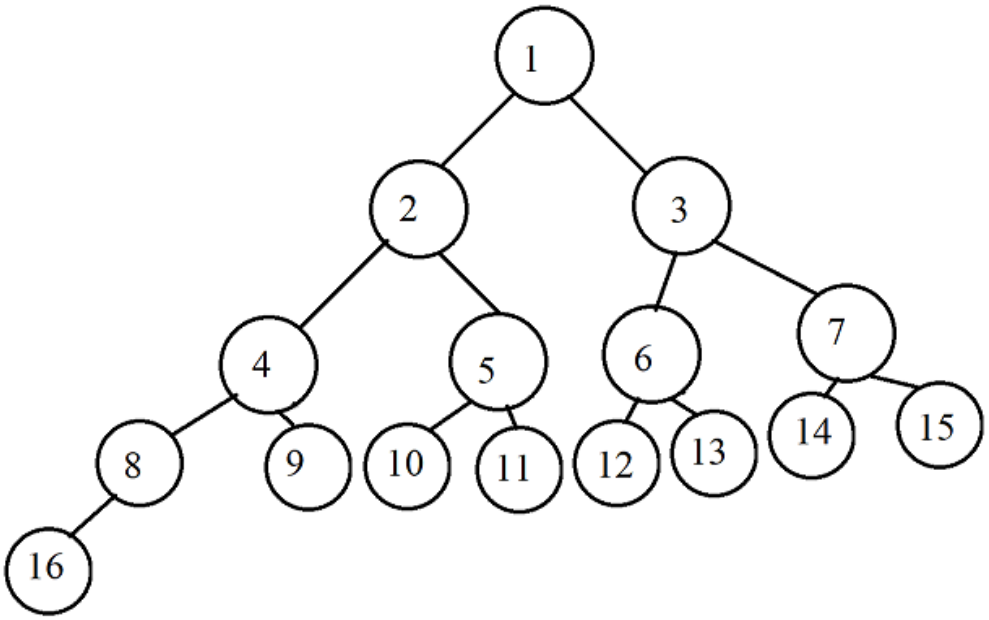
7.c

A h mélységű bináris fák között nyilván a teljes bináris fának van a legtöbb csúcsa. Mivel erre $n = 2^{h+1} - 1$, tetszőleges bináris fára
$n < 2^{h+1}$. Innét $n > 0$ miatt $\log n \ < \log 2^{h+1} = h + 1, amiből \lfloor \log n \rfloor \leq h$ közvetlenül adódik.
Tehát tetszőleges majdnem teljes bináris fára teljesül az egyenlőség.

7.d

A majdnem teljes fa feltétele:
olyan teljes bináris fa, melynek legutolsó levél szintjéről elhagyhatunk néhány levelet (de nem az

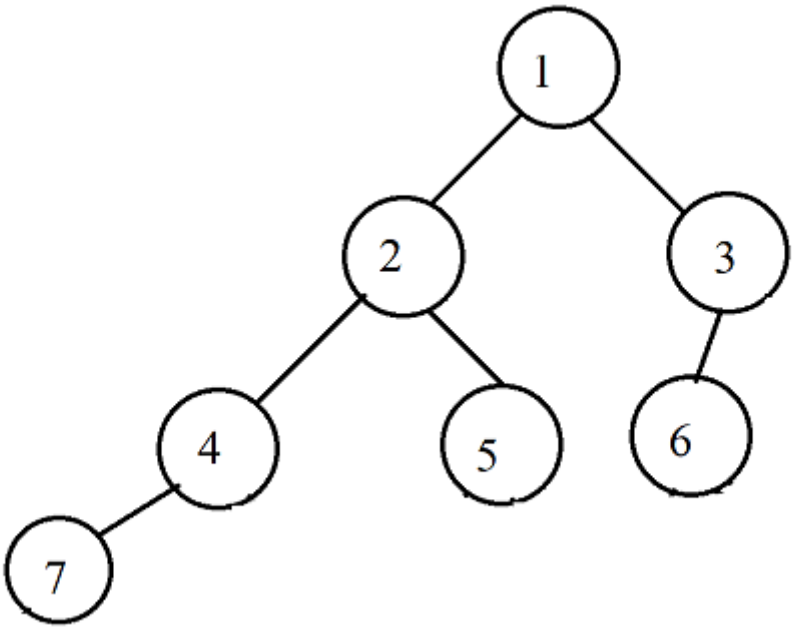
összeset). Vegyünk egy példát:
legyen n = 16 vagyis 2^4. A legkisebb egész szám, amely nagyobb vagy egyenlő 4-nek az a 4, ami $\lfloor \log 16 \rfloor = 4$
Bináris fa:



tehát ha n= 2^h-val akkor az állításunk igaz.

7.e

Vegyünk egy példát:
Legyen n=7 a magassága, azaz h= 2 és rendezzük el a bináris fát az alábbi módon:



Itt a bináris fa nem teljes, mivel az utolsó előtti szinten hiányzik egy csomópont. Azonban a magassága h=2 és $\lfloor \log 7 \rfloor = 2$, ami azt jelenti, hogy a bináris fa megfelel a feltételnek.

8.a -> 1.a-nál

8.b

Bináris keresőfa inorder bejárása egy növekvően rendezett vektort ad. Ez a bináris keresőfa definíciójára vezethető vissza.

8.c –

9.a > 1.a-nál

9.b -> 1.b-nél

9.c -> 1.c-nél

10.a.

Tetszőleges csúcs kulcsánál a bal részfájában minden csúcs kulcsa kisebb **vagy egyenlő**, a jobb részfájában minden csúcs kulcsa nagyobb **vagy egyenlő**.

10.b -> 1.b-nél

10.c –

6. Rendezés lineáris időben

6.1 Edényrendezés

1.a

Algo GY 10.GY-nál alján a táblázat. A beszúrás táblázattal az összefésülés mergesortosan levezetve (felbontod, összelegozod stb.)

1.b

bucket_sort(&L : E1*)

q:=L; n := length(q)
B : E1*[n] //edények listáinak első elemére mutató pointerek tömbje
j = 0 to n-1
B[j] := 0
q ≠ 0
p, q := q, q->next
j:= ⌊n * p->key⌋
p->next:=B[j]; B[j]:=p //lista elejére fűzünk
j = n-1 downto 0
sort(B[j]); q := append(B[j],q) // a B[j] listát a q lista elé fűzi
L := q

1.c

mT(n)∈ $\Theta(n)$

AT(n)∈ $\Theta(n)$, ha egyenletes a feloszlás

MT(n) ∈ $\Theta(n)$, ha összefésülő rendezést használunk

2.a -> 1.a

2.b

mT(n)∈ $\Theta(n)$

MT(n)∈ $\Theta(n^2)$

Mert a beszúró rendezés maximális műveletigénye:
MT(n)∈ $\Theta(n^2)$

AT(n)∈ $\Theta(n)$

2.c

Ha beszúró rendezés helyett összefésülő rendezést használunk.

2.d

Stabil egy rendezés, ha nem változtatja meg az egyenlő kulcsú elemek egymáshoz viszonyított sorrendjét.

2.e

A bucketSort stabil, ha a rendezéshez összefésülő vagy beszúró rendezést használunk.

3.

1.b-s stuki csak a sort helyére mergesort-ot írunk.

6.2 Leszámláló rendezés (counting sort)

1.a

A leszámláló rendezés előfeltétele, hogy a rendezendő adatokat egy tömb tartalmazza.

counting_sort(A, B : ℕ[n]; r, k : ℕ)

```
C : ℕ[r]
        i := 0  to  r-1
    C[i] := 0
        i := 0  to  n-1
    C[digit(k, r, A[i])]++
        i := 1  to  r-1
    C[i] := C[i] + C[i - 1]
    i := n-1 downto 0
    j := digit(k, r, A[i])
    C[j]--
    B[C[j]] := A[i]
```

Műveletigény:
$\Theta(n + r)$, feltételezve, hogy $r \in O(n)$, $\Theta(n + r) = \Theta(n)$ azaz T(n) ∈ $\Theta(n)$

1.b

	30	20	11	22	23	13	$\sum$	13	23	22	11	20	30
0							0						
1			1			2	2	1			0		
2		1		2	3		5		4	3		2	
3	1						6						5

1.c

bemenetre: ugyanabban a számrendszerben, ugyanannyi a számjegyek száma.
rendezésre: a második számjegyre már rendezve volt, ami azt jelenti, hogy a kulcsfüggvény lefutott már egyszer a jobb számjegyre.

6.3 Radix rendezés leszámláló rendezéssel

1.a

Radix csak a leszámláló rendezéses táblázattal

1.b

$\Theta(d * (n + r))$, de a leszámláló rendezés miatt csak $\Theta(n + r)$

1.c

A számjegyek szerinti rendezés és ezt úgy biztosítjuk, hogy lefuttatjuk a leszámláló rendezést a legkevésbé szignifikáns számértéktől a legszignifikánsabb számértékig.

2.a -> 1.a-nál leírtam

2.b

$\Theta(d * (n + r))$, ami, ha d konstans és $r \in O(n)$, akkor a $T_{radix_sort}(n) \in \Theta(n)$

2.c -> 1.c

6.4 Radix rendezés szétválogatással

1.a

Az Algo GY GY10-es pdf 2.oldala

1.b -> 6.3-nál a 2.b feladatnál már leírva

1.c -> 6.3-nál a 2.c feladatnál már leírva

2. 1.a-ra hasonlóan

7. Hasító táblák

7.1 Ütközés feloldása láncolással

1.a. Ezt nem találtam sehol csak az insert-et, ami más

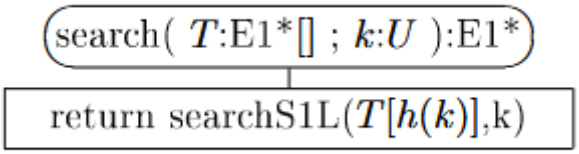
1.b

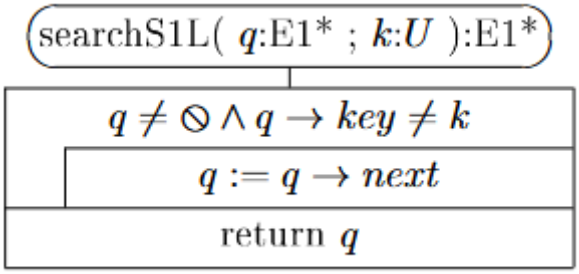
$\alpha = \frac{n}{m}$, ahol n a betöltött helyek száma és m az összes helyek száma

1.c

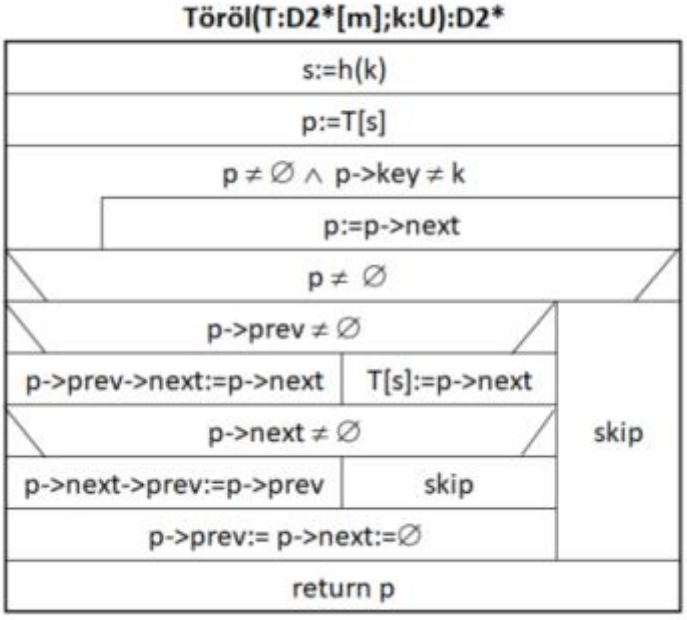
Algo GY GY11 pdf lineáris próba

2.a





2.b



2.c

1.b-nél leírtak +
mT∈ $\Theta(1)$,MT(n) ∈ $\Theta(n)$,AT(n,m) ∈ $\Theta\left(1+\frac{n}{m}\right)$
AT(n,m) ∈ $\Theta\left(1+\frac{n}{m}\right)$ feltétele, hogy a h: U-> 0..(n-1)

7.2 Nyílt címzés

1.a A kitöltöttségi hányados: $\alpha = \frac{n}{m}$, ahol n a betöltött helyek száma és m az összes helyek száma

próbasorozat:

$$h : U \times 0..(m-1) \to 0..(m-1) : \text{próbafüggvény}$$
$$\langle h(k,0), h(k,1), \ldots, h(k,m-1)\rangle : \text{potenciális próbasorozat}$$

Egyenletes hasítás:

A h : U → [0..m) függvény egyszerű egyenletes hasítás, ha a kulcsokat a rések között egyenletesen szórja szét, azaz hozzávetőleg ugyanannyi kulcsot képez le az m rés mindegyikére.

sikertelen keresés, sikeres beszúrás:
$$\frac{1}{1-\alpha}$$

sikeres keresés, sikertelen beszúrás:
$$\frac{1}{\alpha} * \ln\frac{1}{1-\alpha}$$

1.b

A sikertelen keresés hossza 80% kitöltöttségre: 5
Kevesebb, mert kitöltöttebb a tömb így a sikeres keresésre nagyobb az esély.

1.c Algo GY GY11 pdf lineáris próba

2.a -> 1.a

2.b -> 1.b-hez hasonlóan

2.c Algo GY GY11 pdf kettős hasítás

3.a Algo GY GY11 pdf kettős hasítás

3.b

beszúrás:
Ha foglalt, akkor átlépjük
Ha találunk üres rést akkor elhelyezzük, kivéve, ha előbb találtunk törölt rést, mert akkor annak a helyére szúrjuk be a kulcsot.
törlés:
foglalt és sikeres keresés esetén: töröltre állítjuk a rést
Mivel a törlésnél a keresést használjuk, ezért, ha üres rést találunk akkor sikertelen a keresés és így a törlés is.
keresés:
törölt réseket és azokat a réseket, ahol a kulcs nemazonos azt átlépjük,

ha ürest rést találunk akkor a keresés sikertelen.
Ha foglalt rést találunk és a kulcs megegyezik akkor a keresés sikeres

3.c

$\alpha = \frac{n}{m}$, ahol n a betöltött helyek száma és m az összes helyek száma

mT∈ $\Theta(1)$,MT(n) ∈ $\Theta(n)$,AT(n,m) ∈ $\Theta\left(1 + \frac{n}{m}\right)$
AT(n,m) ∈ $\Theta\left(1 + \frac{n}{m}\right)$ feltétele, hogy a h: U-> 0..(n-1)

4.a Algo GY GY11 pdf kettős hasítás

4.b 3.b-nél leírva